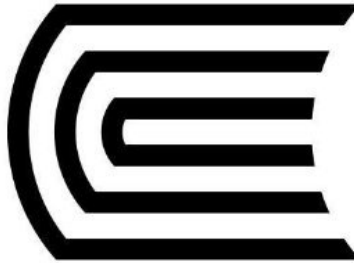


“Año de la recuperación y consolidación de la economía peruana”



Universidad Continental

FACULTAD DE INGENIERÍA

**ESCUELA ACADÉMICO PROFESIONAL DE INGENIERÍA DE SISTEMAS E
INFORMÁTICA**

**"Clasificación y Diagnóstico de Enfermedades de Cultivos a Partir de Imágenes de Hojas, utilizando
Aprendizaje Profundo (CNN) en la Región Cusco, 2025"**

Docente:

- Americo Estrada Sanchez

Desarrollado

- Calderon Romero, Yersson
- Guevara Valdivia, Alejandro Humberto
- Ochoa Cutipa Edwards Andree

Cusco, 2025

Índice de Contenido

1. INTRODUCCIÓN.....4

Índice de Tablas

Índice de Diagramas

RESUMEN

Este proyecto tiene como objetivo desarrollar un modelo de red neuronal convolucional (CNN) para la detección temprana de enfermedades foliares en cultivos andinos, como papa, maíz y tomate, utilizando visión por computadora. La metodología aplicada para el desarrollo fue Scrum, integrando los principios de PMBOK, lo que permitió organizar el trabajo en sprints semanales, priorizando las historias de usuario y alineando el progreso con los objetivos establecidos. Se entrenó un modelo en el framework PyTorch, utilizando un dataset balanceado con imágenes reales de hojas sanas y enfermas, alcanzando una precisión superior al **90 %** en las pruebas de validación. La solución incluye una interfaz web desarrollada con Flask y HTML/CSS, que permite a los agricultores subir imágenes de sus cultivos y recibir diagnósticos en tiempo real sobre el estado fitosanitario de las hojas, junto con sugerencias de tratamiento. Además, el sistema es capaz de procesar imágenes en dispositivos de bajo rendimiento, asegurando su accesibilidad en áreas rurales. El modelo fue evaluado mediante métricas de precisión, recall y F1-score, y se implementó una matriz de confusión para evaluar su desempeño. Este sistema no solo ofrece una herramienta práctica para la detección temprana de enfermedades, sino que también mejora la seguridad alimentaria y la sostenibilidad agrícola en la región Cusco, apoyando a los agricultores en la toma de decisiones informadas para optimizar la gestión de sus cultivos.

1. INTRODUCCIÓN

1.1. Antecedentes

El estudio realizado en el Indian Institute of Technology Kharagpur por Pal et al. (2025) [1] se enfocó en crear un modelo de red neuronal convolucional (CNN) llamado Mob-Res. Este es liviano y sencillo de interpretar y su objetivo es optimizar la identificación de enfermedades en hojas de cultivos. Se utilizó un enfoque experimental en la investigación, que incorporó técnicas de aprendizaje residual a la arquitectura MobileNetV2, y se evaluó con dos extensos conjuntos de datos: Expertos en enfermedades de plantas (con 199,644 imágenes de 58 clases) y PlantVillage (con 54,305 imágenes de 38 clases). Los resultados fueron excepcionales, con una precisión del 97.73 % y del 99.47 %, respectivamente, excediendo incluso a modelos más sofisticados como ViT-L32. Asimismo, los autores incluyeron herramientas de interpretación como Grad-CAM y LIME. En síntesis, el estudio demostró que es posible construir modelos de inteligencia artificial eficientes, precisos y comprensibles que funcionen en dispositivos de bajo costo, lo cual coincide con el propósito de este proyecto al buscar soluciones accesibles para la detección temprana de enfermedades en cultivos andinos mediante técnicas de aprendizaje profundo.

En la École Polytechnique Fédérale de Lausanne (Suiza), Mohanty et al. (2016) [2] llevaron a cabo un estudio con el objetivo de utilizar métodos de aprendizaje profundo para detectar enfermedades en cultivos mediante fotografías de hojas, aprovechando el uso de teléfonos móviles inteligentes y los progresos en visión por computadora. Los escritores utilizaron un compendio de 54,306 imágenes provenientes de 14 especies y 26 clases de enfermedades para entrenar una red neuronal convolucional, la cual llegó a una exactitud del 99.35%. Su método cuantitativo y experimental confirmó la efectividad del modelo para detectar enfermedades de forma automatizada. Determinaron que la combinación del aprendizaje profundo con la captura de imágenes móviles tiene el potencial de ser una herramienta efectiva y accesible para los agricultores, un descubrimiento que apoya a este proyecto enfocado a la detección temprana de enfermedades en cultivos andinos mediante redes profundas (CNN).

Así mismo en la investigación de Pacal et al. (2024) [3], que fue divulgada en Artificial Intelligence Review, proporciona una revisión sistemática acerca de la aplicación del aprendizaje profundo para la identificación de enfermedades en las plantas. Examinó 160 estudios de entre el año 2020 y el 2024, los cuales se enfocan en clasificar, detectar y segmentar enfermedades foliares. Los escritores observaron que las redes neuronales convolucionales logran un nivel de precisión elevado cuando se entrenan con datos equilibrados y de buena calidad. Llegaron a la conclusión de que el aprendizaje profundo es una herramienta efectiva y sostenible para incrementar la productividad agrícola, aportando pruebas que apoyan este proyecto orientado hacia la identificación precoz de enfermedades en cultivos andinos a través de modelos CNN.

Por último, la investigación de Krishna et al. (2025) [4], realizada en la Universidad Trent de Nottingham (Reino Unido), buscó optimizar la identificación de enfermedades en las hojas de plantas utilizando un método multidataset con aprendizaje profundo. Se emplearon modelos CNN como DenseNet201, ResNet50 y EfficientNet-B0, fusionando información del conjunto PlantDoc con imágenes obtenidas de la web y utilizando el aumento de datos con ruido gaussiano. En diversas patologías, el modelo logró una exactitud del 80.19 % y F1-scores que sobrepasan el 90

%.

Los autores llegaron a la conclusión de que incorporar diversas fuentes de datos fortalece la habilidad del modelo para ajustarse a situaciones reales, brindándole un soporte sólido a este proyecto orientado hacia cultivos andinos en medio de escenarios ambientales cambiantes.

1.2. Identificación y formulación del problema

Las plagas y las enfermedades agrícolas constituyen una de las más graves amenazas para la seguridad alimentaria en el mundo. La Organización de las Naciones Unidas para la Alimentación y la Agricultura (FAO) estima que, como consecuencia de estos factores, se pierde hasta el 40% del total de cultivos a nivel mundial anualmente; esto ocasiona mermas financieras por encima de los 220 mil millones de dólares y empeora las circunstancias de pobreza y desprotección entre los agricultores en naciones en desarrollo [5]. La circunstancia actual pone de manifiesto la urgencia de crear soluciones tecnológicas que permitan identificar las enfermedades vegetales a tiempo y disminuir el efecto que estas tienen en los sistemas de producción.

Savary et al. (2019) [6], por su parte, hicieron un estudio a nivel mundial sobre la carga de plagas y patógenos en los cultivos alimentarios más importantes, y llegaron a la conclusión de que dolencias como la mancha foliar, el tizón o la roya disminuyen la productividad entre un 10 % y un 40 %, dependiendo del clima y de la especie vegetal. Los escritores enfatizan que la falta de herramientas para el diagnóstico rápido en campo es uno de los principales impedimentos para manejar estas dolencias, sobre todo en áreas agrícolas con poca tecnología.

Por otro lado, a nivel nacional, Leon Betsabe et al. (2022) [7] llevaron a cabo una investigación acerca de las infecciones y plagas más frecuentes en cultivos de cañihua del altiplano peruano, y hallaron que la presencia de enfermedades foliares e insectos que impactan directamente en el rendimiento muestra variaciones significativas. Su estudio demostró que, debido a la escasez de tecnologías asequibles para el diagnóstico precoz y de personal técnico especializado, la supervisión y el control en estas áreas son limitados.

De acuerdo con el Censo Nacional Agropecuario, la producción de papa en la región Cusco abarca cerca de 37 000 hectáreas, lo que la hace uno de los principales pilares de la producción agrícola andina [8]. Sin embargo, según ha determinado el Servicio Nacional de Sanidad Agraria (SENASA), hay plagas que perjudican los cultivos de papa en provincias cusqueñas como Urubamba, Paruro y Anta: la rancha (*Phytophthora infestans*), la polilla (*Phthorimaea operculella*) y el gorgojo andino (*Premnotrypes* spp.) [8]. Estas condiciones se manifiestan de manera constante a lo largo del año, y aunque el sistema de vigilancia fitosanitaria realiza muestreo y diagnóstico, una gran cantidad de productores no tienen acceso inmediato a instrumentos tecnológicos para identificar con anticipación los síntomas foliares. Esta realidad evidencia que, a pesar de los intentos institucionales de control, sigue existiendo una considerable brecha en el diagnóstico accesible, rápido y ajustado al contexto andino. Esto impacta directamente en la eficacia productiva, en los costos del control y en la seguridad alimentaria de los productores agrícolas locales.

El diagnóstico de la situación agrícola en la región cusqueña muestra que aumenta la vulnerabilidad ante el surgimiento y difusión de enfermedades en cultivos andinos como el maíz, la quinua y la papa. Según informes del Servicio Nacional de Sanidad Agraria (SENASA), la existencia de plagas como la polilla de la papa y la ranca ha causado que el rendimiento disminuya notablemente, lo cual impacta directamente en la economía de las poblaciones rurales [8]. Además, la habilidad técnica restringida de los agricultores para detectar visualmente los primeros signos de las enfermedades contribuye a que las medidas de control se tomen con retraso y a que los brotes sean más graves. Esta carencia en el diagnóstico, que también se ha visto en investigaciones anteriores sobre la oscilación de plagas en sembrados altoandinos [7], indica que el procedimiento de identificación manual de enfermedades es ineficaz y está supeditado a la experiencia empírica del agricultor.

El impacto social y económico se volverá más serio si no se ponen en práctica tácticas de detección automatizada y temprana. Como alerta la FAO, que estima pérdidas anuales de hasta el 40 % de las cosechas en todo el mundo debido a causas fitosanitarias [5], las enfermedades podrían disminuir significativamente la producción regional, poner en riesgo la seguridad alimentaria y provocar pérdidas financieras difíciles de reponer. En un entorno rural en el que la agricultura es la fuente principal de ingresos, si no se cuenta con herramientas tecnológicas para hacer diagnósticos a tiempo, esto podría perpetuar un ciclo de escasa productividad, dependencia de deterioro del medio ambiente. Así, la implementación de tecnología a través de modelos de aprendizaje profundo se vuelve una necesidad apremiante para optimizar el control de enfermedades y asegurar que la agricultura sea sostenible en la región andina.

Si no se implementan estrategias adecuadas para la detección temprana de enfermedades en los cultivos, las consecuencias serán aún más perjudiciales tanto a nivel social como económico. La falta de herramientas tecnológicas adecuadas puede llevar a la disminución de la productividad agrícola, lo que pone en riesgo la seguridad alimentaria y genera grandes pérdidas económicas. En regiones rurales donde la agricultura es la principal fuente de ingresos, la ausencia de diagnósticos oportunos puede perpetuar un ciclo de baja producción y generar una mayor dependencia de prácticas que deterioran el medio ambiente. Por lo tanto, el uso de tecnologías como los modelos de aprendizaje profundo es crucial para optimizar la gestión de enfermedades en los cultivos y garantizar una agricultura más sostenible en las zonas andinas.

1.3. MARCO TEÓRICO

1.3.1. Bases teóricas

1.3.1.1. Inteligencia Artificial y Aprendizaje Profundo

1. Inteligencia Artificial (IA)

La inteligencia artificial es una disciplina de la informática que se dedica a crear sistemas capaces de razonar, tomar decisiones independientes y percibir su entorno en actividades que normalmente se necesitarían de la inteligencia humana [9]. Estos sistemas utilizan algoritmos, datos y hardware para aprender patrones y actuar sin intervención constante; tienen aplicaciones que van desde la visión por computadora hasta el procesamiento del lenguaje natural [10].

2. Aprendizaje Profundo (Deep Learning)

La técnica de aprendizaje profundo, que es avanzada y forma parte de la IA, organiza algoritmos en capas jerárquicas que aumentan su complejidad, lo cual posibilita que la máquina vaya aprendiendo representaciones de datos cada vez más abstractas [11]. Estas redes, a diferencia del aprendizaje automático convencional, profundizan en varios niveles para captar patrones complejos; por ejemplo, primero reconocen formas y colores en las imágenes y después objetos enteros [11].

3. Red Neuronal Convolutiva (CNN)

Una red neuronal convolutiva (CNN) es un tipo de red de aprendizaje profundo que se ha creado específicamente para manejar datos estructurados en forma de cuadrícula, como las imágenes. Esto lo realiza a través de una combinación de capas completamente conectadas, capas de agrupamiento ("pooling") y capas de convolución [12]. Estas redes, mediante la extracción progresiva de rasgos que van desde bordes y texturas hasta formas complejas, simplifican las tareas de reconocimiento visual y clasificación [13].

4. Arquitectura de red (capas convolucionales, pooling, fully connected)

Una CNN tiene una estructura típica que consta de tres capas principales: la capa convolutiva, que emplea filtros para obtener características locales; la capa pooling, que disminuye las dimensiones espaciales con el fin de optimizar los cálculos y mejorar la generalización; y la capa fully connected, que combina las propiedades aprendidas para generar la última predicción [12][14]. Cada componente cumple una función: extracción, reducción y decisión, respectivamente; esto posibilita que se construyan modelos eficaces y exactos.

5. Transfer Learning (Aprendizaje por Transferencia)

La transferencia de aprendizaje es un método que consiste en adaptar un modelo previamente entrenado, para que pueda resolver una tarea diferente pero vinculada, lo cual hace posible que la generalización mejore y el tiempo de entrenamiento para la nueva tarea disminuya [15]. Esto es especialmente práctico en situaciones donde no se dispone de muchos datos o son muy costosos de conseguir, como en el caso de detectar enfermedades en plantas.

6. Entrenamiento del modelo

El entrenamiento de un modelo de aprendizaje profundo implica presentarle un conjunto de datos representativos (dataset) para que modifique sus pesos con el objetivo de reducir una función de pérdida.

Es fundamental eludir el sobreajuste (overfitting) durante este proceso, ya que se produce cuando el modelo aprende características específicas del dataset de entrenamiento y pierde la capacidad de generalizar [12]. Técnicas como la validación cruzada, la regularización y el empleo apropiado de conjuntos de prueba permiten controlar este fenómeno.

7. Métricas de evaluación

Las métricas de evaluación son indicadores cuantitativos fundamentales que posibilitan la valoración del rendimiento de un modelo de clasificación. Estas incluyen la puntuación F1, la precisión (accuracy) y la recuperación (recall), las cuales proporcionan una perspectiva más integral que el mero conteo de aciertos [16]. En situaciones de clases desbalanceadas, por ejemplo, una precisión elevada puede resultar engañosa porque el modelo podría predecir siempre la clase predominante y aún así mostrar un "accuracy" elevado. En contraposición, el F1-score integra la precisión y el recall para mostrar de una manera más precisa la habilidad del modelo para predecir correctamente tanto para las clases negativa como positiva [16]. Estas métricas, junto con el AUC y la curva ROC, son cruciales para corroborar que el modelo generalice correctamente en lugar de estar simplemente sobreajustado.

8. Matriz de confusión

La matriz de confusión es una tabla que sirve para comparar las predicciones de un clasificador con los valores reales de una muestra, y así categorizar los resultados en verdaderos positivos (TP), verdaderos negativos (TN), falsos positivos (FP) y falsos negativos (FN) [17]. Además de proporcionar una percepción clara de los errores que está cometiendo el modelo, esta herramienta es esencial para realizar cálculos más complejos en el futuro, como la precisión o el recall. Para las tareas multiclase, la matriz se amplía para que se puedan ver las confusiones entre múltiples etiquetas, lo cual hace más sencillo el análisis y la optimización del modelo [17].

1.3.1.2. Agricultura y Fitopatología

1. Fitopatología (estudio de enfermedades de plantas)

La fitopatología es la rama de la ciencia que se ocupa del análisis de las enfermedades que impactan a las plantas, incluyendo los factores abióticos y los agentes infecciosos (como virus, hongos, bacterias o nematodos) [18]. Esta ciencia estudia el proceso por el que la planta se enferma, los mecanismos de interacción entre agente y hospedante, además de las tácticas para su diagnóstico y control preventivo [19].

2. Síntomas foliares

Las alteraciones funcionales o estructurales de las hojas de las plantas, que se producen cuando están sometidas a estrés o enfermedades, son los síntomas foliares. Las manchas, la necrosis, las deformaciones y la decoloración son ejemplos de estos [20]. Estos signos posibilitan que agricultores y fitopatólogos detecten de manera temprana los cambios en el cultivo antes de que los daños se propaguen a otras áreas de la planta.

1.3.1.3. Procesamiento de Imágenes

1. Visión por Computadora (Computer Vision)

La visión por computadora es una especialidad de la inteligencia artificial que posibilita que las máquinas analicen y comprendan el contenido de imágenes o vídeos, no solo alterarlos, sino también obtener información valiosa acerca de lo que estos muestran [21]. Esta tecnología se utiliza en el ámbito agrícola para identificar visualmente signos de salud, enfermedad o daño en las plantas a través de algoritmos que imitan la percepción humana.

2. Preprocesamiento de imágenes

El preprocesamiento de imágenes es la etapa preliminar del análisis visual. En ella se llevan a cabo labores como el aumento de datos (modificaciones de color, rotaciones, flips), la normalización, el redimensionamiento y la supresión de ruido para optimizar la calidad con que se ingresan los datos al modelo y garantizar que el aprendizaje sea más sólido y aplicable en general [21].

3. Clasificación de imágenes

La categorización de imágenes es el procedimiento por el cual un sistema de visión computacional asigna una categoría o etiqueta a una imagen entera, tomando como base su contenido visual [22]. Este método es común en trabajos que necesitan identificar patologías de las hojas, porque el sistema vincula cada imagen con una de las categorías establecidas (por ejemplo, "sana" o "enferma") tras haber sido adiestrado con ejemplos previamente etiquetados.

1.4. OBJETIVOS DEL PROYECTO

1.4.1. OBJETIVO GENERAL

OG: Desarrollar un modelo de red neuronal convolucional (CNN) entrenado con imágenes de hojas de cultivos andinos, como papa, maíz y tomate, que permita detectar de forma temprana las enfermedades foliares, ofreciendo diagnósticos confiables y visuales para apoyar la toma de decisiones de los agricultores en la región Cusco.

1.4.2. OBJETIVOS ESPECÍFICOS

1. OE1: Implementar un modelo de aprendizaje profundo (CNN) utilizando el framework PyTorch, capaz de clasificar diferentes tipos de enfermedades en hojas de cultivos andinos.
2. OE2: Entrenar y validar el modelo con un dataset balanceado compuesto por imágenes reales de hojas sanas y enfermas, asegurando una adecuada generalización y minimizando el sobreajuste.
3. OE3: Diseñar una interfaz web interactiva que permita al usuario subir imágenes y obtener en tiempo real el diagnóstico del estado fitosanitario de la hoja.
4. OE4: Evaluar el rendimiento del modelo mediante métricas de precisión, recall, F1-score y matriz de confusión, garantizando resultados interpretables y confiables.

2. CONOCIMIENTOS DE INGENIERÍA APLICADOS / RELACIONADOS

Este proyecto que diagnóstica y clasifica las enfermedades que existen en las hojas de los cultivos mediante las redes neuronales convolucionales o (CNN) esto entrelaza diversos conocimientos fundamentales en la ingeniería. Estos conocimientos hacen posible la implementación, en el desarrollo de una solución tecnológica que sea precisa y más que todo adaptable a la realidad agrícola del Cusco y con capacidad para escalar. Cada campo del conocimiento aporta desde un ámbito particular, coordinando para tratar una cuestión concreta que impacta a los agricultores que no disponen de herramientas contemporáneas de diagnóstico y monitoreo. Esta información no solo posibilita la implementación del sistema, sino también garantiza que su desempeño satisfaga de manera efectiva los requerimientos del campo peruano.

- **Aplicación/Relación en Matemática**

Las matemáticas aplicadas son una de las piedras angulares del proyecto. La identificación automatizada de enfermedades en imágenes se basa en operaciones de álgebra lineal, cálculo diferencial y estadística. Las convoluciones, la base de las CNN, son multiplicaciones matriciales que reconocen patrones visuales, y la retropropagación usa derivadas parciales para ajustar los pesos. Agregando esto, la estadística nos permitirá conocer mejor las distribuciones de los datos, después calcular medidas de rendimiento y comprobar cómo realmente se generaliza el modelo ante variedad de datos nuevos. Dichas herramientas harán que la red neuronal se entrene de una forma sólida y sin distorsiones, y lo más importante que funcione correctamente con imágenes reales que serán capturadas en el campo en condiciones no controladas.

- **Aplicación/Relación en Machine Learning**

El aprendizaje automático (machine learning) constituye otra base esencial del proyecto, ya que ofrece el marco conceptual que permite que un sistema aprenda directamente a

partir de ejemplos. En el contexto agrícola del Cusco, donde enfermedades como el tizón tardío en papa, la roya amarilla y las manchas foliares pueden causar pérdidas significativas, el machine learning permite automatizar un proceso que tradicionalmente depende de la experiencia del agricultor o de la disponibilidad de especialistas. Este conocimiento habilita la creación de modelos capaces de identificar patrones en las hojas que, en muchos casos, son imperceptibles para el ojo humano en las primeras fases de una infección, lo cual aporta un valor enorme para la prevención y la toma de decisiones tempranas.

- **Aplicación/Relación en Deep Learning**

Sobre esta estructura crece el aprendizaje profundo del (deep learning), una de las técnicas principales del proyecto. Ya que a diferencia de los enfoques clásicos el deep learning aprende o puede aprender características jerárquicas de cada imagen sin tener que diseñarlas o crearlas manualmente. Es fundamental en un contexto porque en la misma enfermedad puede tender a manifestarse de manera diferente en función del clima, también la variedad o el grado de infección. Estas redes profundas hacen que identifiquen desde las características más básicas, como los cambios en la textura o el color, hasta los patrones más complejos que se diferencian entre varias enfermedades. Dicha habilidad es lo que permite obtener precisiones superiores al 80% o 85% incluyendo imágenes capturadas por los celulares de diferente calidad.

- **Aplicación/Relación en Redes Neuronales Convolucionales**

La arquitectura que tiene es capaz de analizar imágenes de hojas que son tomadas por agricultores que tienen condición de luminosidad baja, fondos que son variables o también por los ángulos distintos. Sus capas son como extractores automatizados de características, porque tienen la capacidad de identificar síntomas foliares, es decir, manchas, necrosis o alteraciones de su color. Las capas o pooling que están totalmente conectadas terminan de complementar el proceso, reduciendo la complejidad para hacer al mismo tiempo una predicción final. Al elegir CNN sobre otras técnicas que están respaldadas por la literatura científica, también en el estado del arte de aplicaciones como PlantVillage o Plantix, que han demostrado ser tener la mejor técnica para este tipo de problema.

- **Aplicación/Relación en Procesamiento digital de imágenes**

En este caso el procesamiento tiene la posibilidad de la preparación de las fotos para que el modelo obtenga la capacidad analizarlas adecuadamente. Se hace esencial en este proyecto, porque los agricultores en su mayoría tienen la posibilidad de tomar fotos con sus celulares de gama baja o media, en el aire libre y sin tener el control de la luz. Esto hace que el modelo sea más robusto gracias a sus técnicas como la normalización, el destruido, también la segmentación y el incremento de datos por transformaciones, es decir, cortes, rotaciones, cambios en el brillo. Esto permite que el sistema funcione aun incluso en comunidades tienen condiciones tecnológicas escasas, y esto lo hace un componente esencial del enfoque social del proyecto.

- **Aplicación/Relación en Data Science**

En la ciencia de datos, esencialmente con la creación, análisis y administración del conjunto de datos, tiene un rol fundamental, es decir, para la recolección de imágenes auténticas, la categorización de clases y depuración de datos se necesita tener un buen dominio en documentación, exploración y también transformación. La ciencia de datos hace posible asegurar que el conjunto empleado para entrenar la CNN representa las enfermedades que son más frecuentes en cultivos, como la papa, el maíz y tomate, para evitar problemas de desequilibrio que podrían llegar a perjudicar la efectividad del modelo final. Finalmente, en los análisis estadísticos que se realizan, posibilita determinar qué enfermedades son más comunes en cada área, y esto es información útil para entidades de control fitosanitario y más para los agricultores.

- **Aplicación/Relación en Programación en Python**

Un lenguaje de programación como Python es fundamental para este tipo de proyectos, ya que proporciona un entorno amplio de librerías especializadas como OpenCV, TensorFlow, NumPy, PyTorch, etc. Además de otros recursos indispensables para la elaboraciones de modelo y tratamiento de imágenes. El lenguaje Python hace que sea posible la integración de cada fase del sistema, desde el preprocesamiento que va pasando por el entrenamiento y una evaluación, hasta una exportación del modelo para que se pueda utilizar en un servicio web. Asimismo, por esto se emplean instrumentos como Google Colab o Jupyter Notebook para la optimización de la arquitectura y realizar experimentos de forma más interactiva.

3. INGENIERO Y LA SOCIEDAD:

1. Justificación social:

La propuesta tecnológica del proyecto impacta significativamente las comunidades agrícolas de Cusco, donde los agricultores dependen de cultivos como papa, maíz y tomate para su sustento. Al no tener herramientas modernas de diagnóstico hace que la identificación de enfermedades vaya a depender de la experiencia propia del agricultor, dificultando la percepción exacta de los síntomas iniciales y confundirá estos problemas con los daños ambientales. Asimismo, una progresión incontrolable de enfermedades, afectando y dañando la productividad de familias muy vulnerables. Entonces, la implementación del sistema basado en visión por la computadora y las redes neuronales convolucionales ofrecerá a estas comunidades una nueva herramienta que sea accesible, que democratice el acceso al conocimiento para detección temprana de enfermedades y solo era posible con visitas técnicas o análisis de un laboratorio. Como conclusión, este proyecto da como objetivo mejorar la seguridad alimentaria, también disminuir la incertidumbre con la toma de decisiones agrícolas y empoderar a los pequeños productores con un diagnóstico inmediato y confiable.

3.1. Justificación Económica:

Hay pérdidas económicas en el sector agrícola de los Andes peruanos, que son causadas por enfermedades de plantas y esto hace que limiten su crecimiento. La FAO que es una organización de las naciones unidas, estima que hasta un 40% de los cultivos son perdidos anualmente por causa de las plagas y enfermedades. Según la SENASA indican que las enfermedades como el tizón tardío y la roya pueden reducirse significativamente con la producción, afectando a pequeños productores sin los recursos necesarios. Dichas pérdidas tienen un impacto tanto a nivel familiar y regional, lo cual incrementa los precios y esto genera inestabilidad económica. Debido a esto podría resultar un aumento de rendimiento por hectárea, por menores costos más que todo en pesticidas, y así menos necesidad de replantar y mayor estabilidad económica para las familias. Finalmente, la capacidad de poder actualizar el modelo con nuevos datos asegurará que la herramienta sea valiosa y adaptable a largo plazo o a futuro.

3.2. Justificación ambiental:

El presente proyecto tiene un impacto ambiental significativo, porque al permitir la detección rápida y temprana de enfermedades, mejorará la productividad y reducirá la dependencia de pesticidas. Ahora, si se identifica tarde una enfermedad, esto provoca un uso agresivo de agroquímicos que perjudica bastante al suelo, agua y biodiversidad en general. Y un diagnóstico temprano permitirá aplicar estrategias de control mucho más precisas y menos agresivas, porque protege los ecosistemas agrícolas, mantiene la salud del suelo y promueve prácticas sostenibles. La herramienta tecnológica propuesta es clave para fomentar la agricultura responsable en áreas rurales, guiando decisiones informadas y minimizando el impacto negativo de respuestas tardías a enfermedades.

3.3. Acontecimientos tecnológicos y científicos: Relacionados a la solución que influenciaron en la propuesta

Dicho proyecto se fundamenta en recientes avances tecnológicos y científicos que han hecho posible el desarrollo de soluciones viables, eficientes y accesibles. En esta última década, las redes neuronales convolucionales (CNN) crecieron notablemente, respaldadas por Mohanty et al. (2016) y el último o reciente de Pal et al. (2025), demostrando que modelos ligeros basados en CNN nos pueden ofrecer alta y mejor precisión en dispositivos de bajo costo. Añadiendo esto la proliferación de frameworks como PyTorch y TensorFlow facilitó la creación de modelos de aprendizaje profundo, lo cual permite que estudiantes e instituciones sin muchos recursos avanzados entrenen modelos usando herramientas accesibles como Google Colab. Entonces, estas soluciones no se ajustan completamente a zonas andinas ni tienen limitaciones de conectividad en áreas rurales del Perú, lo que sugiere la necesidad de propuestas específicas concretas como la que se presenta en este proyecto.

4. METODOLOGÍA EMPLEADA (De acuerdo al problema)

Se adoptó a trabajar con la metodología ágil Scrum, porque permite trabajar de manera iterativa, flexible y enfocada en la entrega continua de valor a los usuarios finales. Dicho enfoque resulta adecuado para este proyecto que combina componentes tecnológicos complejos como el entrenamiento de modelos de aprendizaje profundo para agricultores y comunidades rurales, ya que los requerimientos pueden mejorarse continuamente a medida que se validan las funcionalidades.

<i>Interesados en el proyecto</i>		
<i>Nombre</i>	<i>Función</i>	<i>Teléfono</i>
Yersson Calderon Romero	Supervisar el proyecto y tomar decisiones estratégicas.	925271857
Alejandro Humberto Guevara Valdivia	Gestionar las actividades diarias y coordinar al equipo.	984582724
Edwards Andree Ochoa Cutipa	Planificar y controlar el cronograma y presupuesto del proyecto.	992133112
Americo Estrada Sanchez	Evaluador del proyecto	-

4.1. APORTES /DESCUBRIMIENTOS: (3 mínimo por PMV)

PMV 1 – Modelo de entrenamiento CNN y procesamiento de imágenes

El modelo logra precisiones arriba del 80% en conjuntos de datos que no sean muy grandes, además aplicar técnicas de aumento de datos disminuyen el sobreajuste y ayudan

un entrenamiento estable porque se llegó a notar que las imágenes que tienen mala luz dañan el funcionamiento del modelo, por eso se añadió un preprocesamiento extra.

PMV 2 – Integración del modelo con la API para la predicción y pruebas de inferencia

Se utilizó la API de predicción usando frameworks como Flask o FastAPI, con el fin de integrar el modelo ya entrenado en un entorno funcional que pueda obtener imágenes y recibir diagnósticos. En el proceso uno de los aportes más resaltantes fue la identificación entre el rendimiento del modelo en un lugar de entrenamiento enfrentándose al rendimiento en la API. Se reveló que debido a cambios en el preprocesamiento puede que haya discrepancias en las predicciones y eso impulsó la necesidad de estandarizar el pipeline de procesamiento en el entrenamiento como en la inferencia.

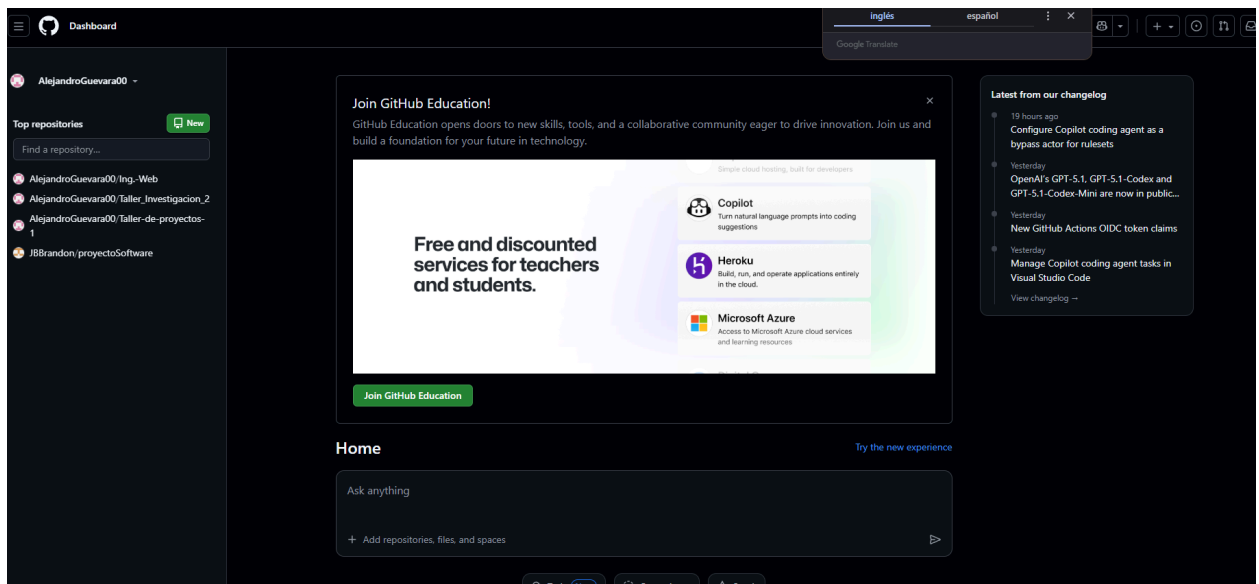
PMV 3 – Interfaz web y pruebas con usuarios para el acceso al diagnóstico

En este caso los agricultores que serán los usuarios aprecian que la interfaz será sencilla y rápida, así que se añadió un historial de diagnósticos para facilitar el seguimiento de enfermedades que tendrán al igual que las pruebas en dispositivos que no sean tan potentes mostrarán que el sistema funcionará aun con pocos recursos.

5. USO DE HERRAMIENTAS MODERNAS

GitHub: Control de versiones, revisiones y despliegue continuo

GitHub, se usará para almacenar código, registrar modificaciones y mantener diversas versiones, asegurando la integridad de los datos..



IDEA-PROYECTO-main > Project_Plant_Disease.ipynb > M4 Importar Dependencias

Generate + Code + Markdown Run All Clear All Outputs Outline

Importar Dependencias

```
[1] import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
[2] import torch
from torchvision import datasets, transforms, models # datasets , transforms
from torch.utils.data.sampler import SubsetRandomSampler
import torch.nn as nn
import torch.nn.functional as F
from datetime import datetime
```

```
[3] #Load_ext nb_black
```

Cargar datos desde drive al interno

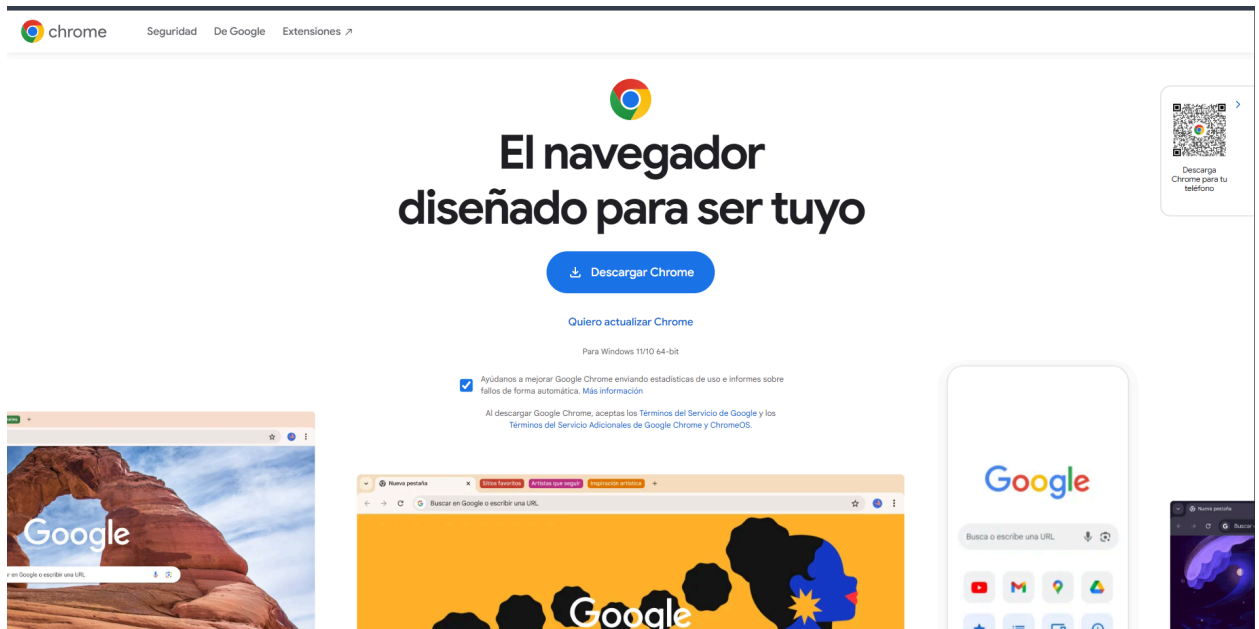
```
[4] from google.colab import drive
drive.mount('/content/drive')
```

```
[5] Mounted at /content/drive
```

```
[6] # 1. descomprime sin -j (conserva carpetas)
!cd /content/Dataset && unzip -o "/content/drive/MyDrive/dataset.zip"

# 2. si se creó la carpeta madre, sube un nivel
!shopt -s dotglob nullglob
!if [ -d "/content/Dataset/Plant_leave_diseases_dataset_with_augmentation" ]; then \
    mv /content/Dataset/Plant_leave_diseases_dataset_with_augmentation/* /content/Dataset/ ; \
    rmdir /content/Dataset/Plant_leave_diseases_dataset_with_augmentation ; \
fi
```

- **Navegador web GOOGLE CHROME):** Para visualizar nuestro desarrollo web

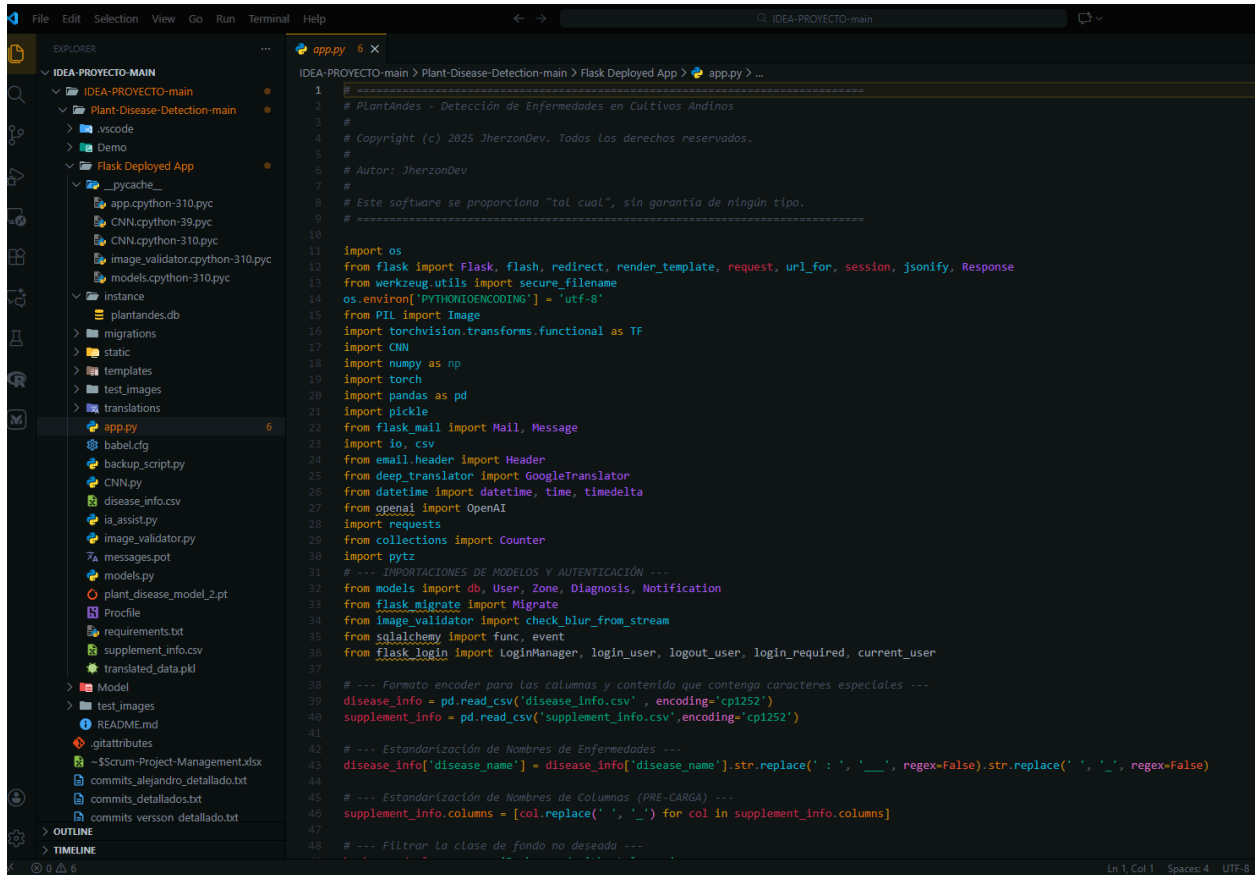


- **PyTorch:** para modelamiento y entrenamiento de redes profundas

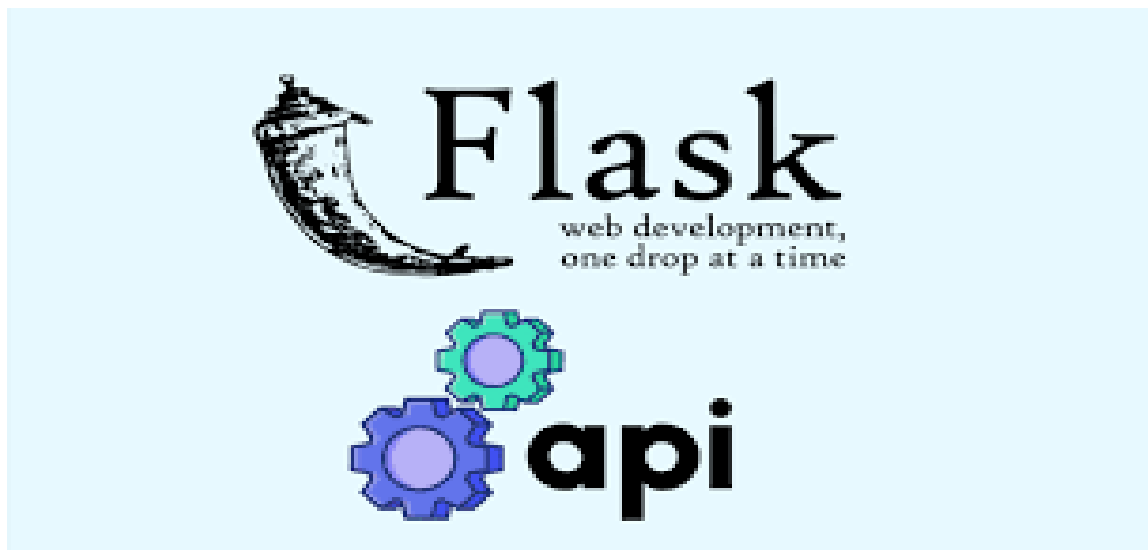
Se optó por PyTorch debido a su flexibilidad y comodidad al trabajar con redes neuronales. Su arquitectura dinámica simplifica la experimentación, permitiendo la modificación y evaluación rápida de modelos.



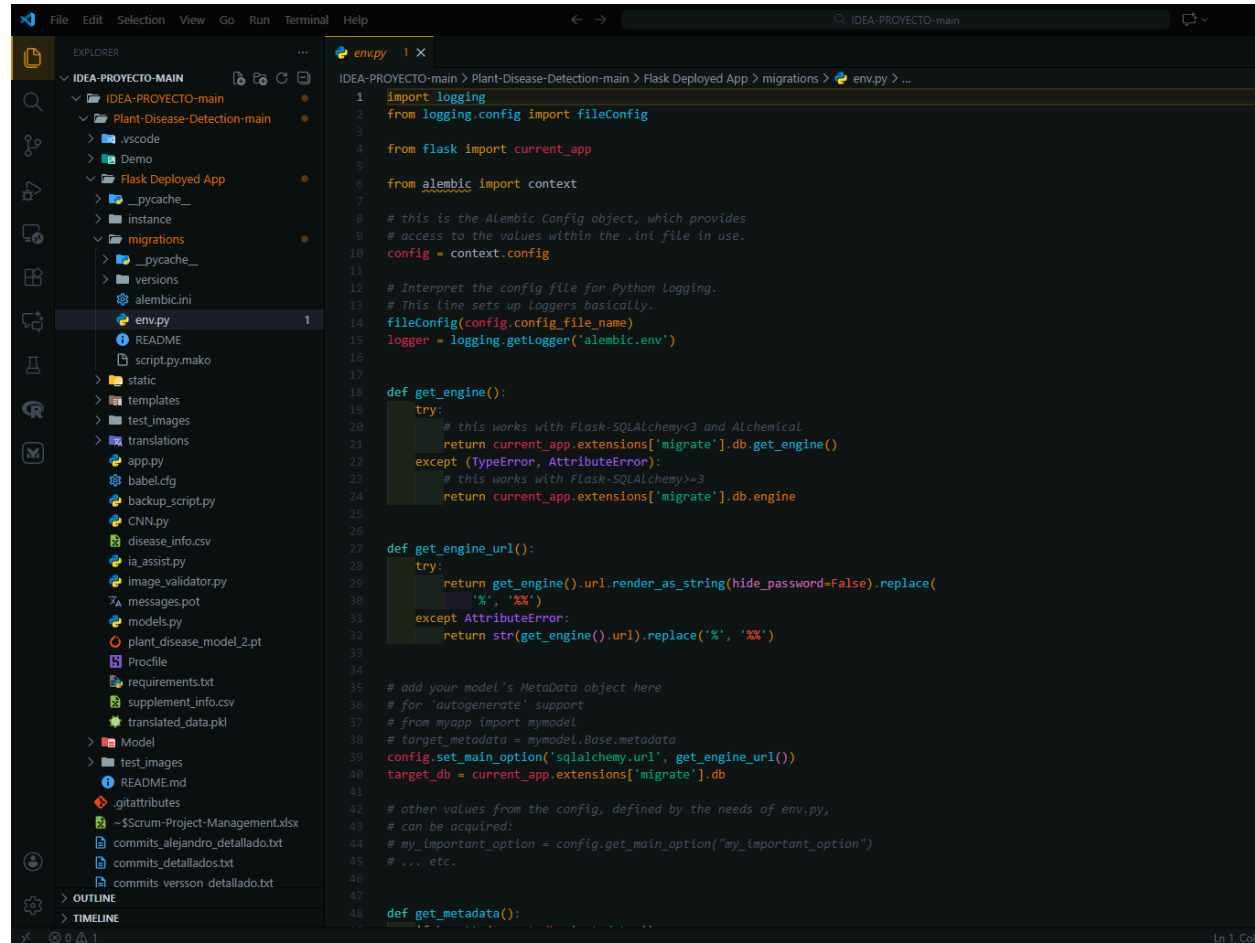
- **FastApi/Flask:** Para implementar servicios web escalables, empleo tanto FastAPI como Flask. Flask es mi opción ideal cuando necesito una solución rápida y sencilla, especialmente para pruebas y prototipos.



```
1 # =====  
2 # PlantAndes - Detección de Enfermedades en Cultivos Andinos  
3 #  
4 # Copyright (c) 2025 JherzonDev. Todos Los derechos reservados.  
5 #  
6 # Autor: JherzonDev  
7 #  
8 # Este software se proporciona "tal cual", sin garantía de ningún tipo.  
9 # =====  
10  
11 import os  
12 from flask import Flask, flash, redirect, render_template, request, url_for, session, jsonify, Response  
13 from werkzeug.utils import secure_filename  
14 os.environ['PYTHONIOENCODING'] = 'utf-8'  
15 from PIL import Image  
16 import torchvision.transforms.functional as TF  
17 import CNN  
18 import numpy as np  
19 import torch  
20 import pandas as pd  
21 import pickle  
22 from flask_mail import Mail, Message  
23 import io, csv  
24 from email.header import Header  
25 from deep_translator import GoogleTranslator  
26 from datetime import datetime, time, timedelta  
27 from openai import OpenAI  
28 import requests  
29 from collections import Counter  
30 import pytz  
31 # --- IMPORTACIONES DE MODELOS Y AUTENTICACIÓN ---  
32 from models import db, User, Zone, Diagnosis, Notification  
33 from flask_migrate import Migrate  
34 from image_validator import check_blur_from_stream  
35 from sqlalchemy import func, event  
36 from flask_login import LoginManager, login_user, logout_user, login_required, current_user  
37  
38 # --- Formato encoder para las columnas y contenido que contenga caracteres especiales ---  
39 disease_info = pd.read_csv('disease_info.csv', encoding='cp1252')  
40 supplement_info = pd.read_csv('supplement_info.csv', encoding='cp1252')  
41  
42 # --- Estandarización de Nombres de Enfermedades ---  
43 disease_info['disease_name'] = disease_info['disease_name'].str.replace(':', '_', regex=False).str.replace(' ', '_', regex=False)  
44  
45 # --- Estandarización de Nombres de Columnas (PRE-CARGA) ---  
46 supplement_info.columns = [col.replace(' ', '_') for col in supplement_info.columns]  
47  
48 # --- Filtrar la clase de fondo no deseada ---  
49
```

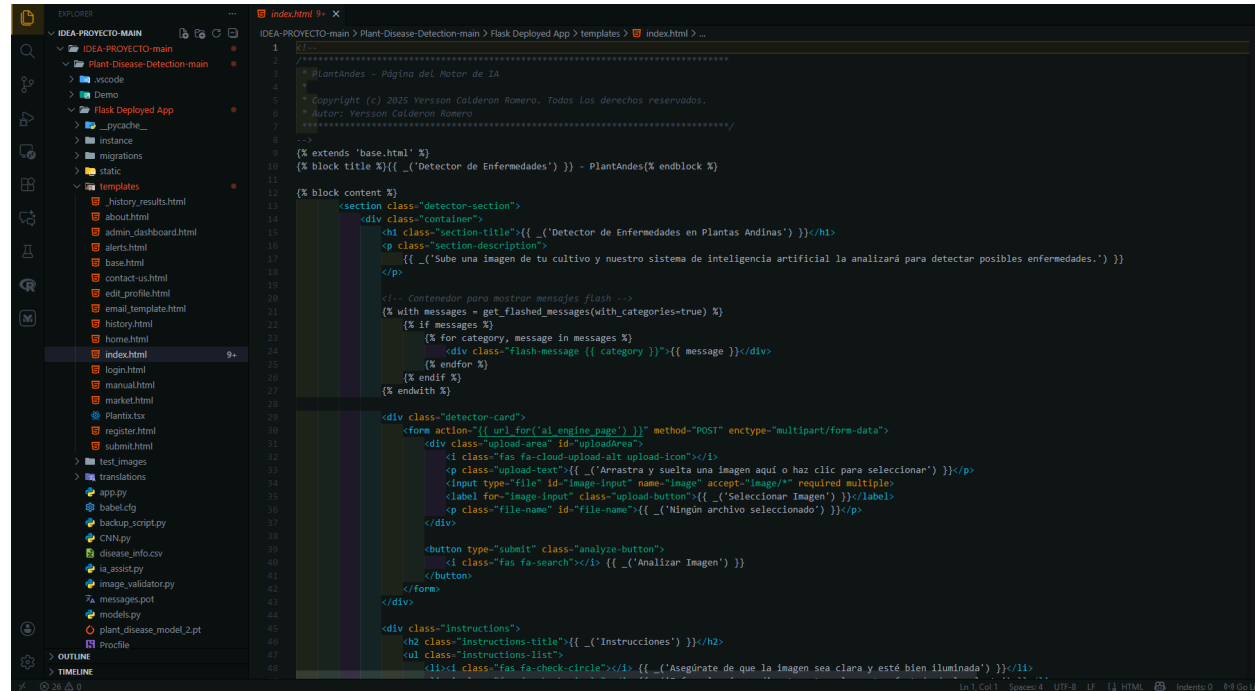


- **PYTHON, HTML, CSS:** Para manejo de datos y para la construcción de una interfaz intuitiva

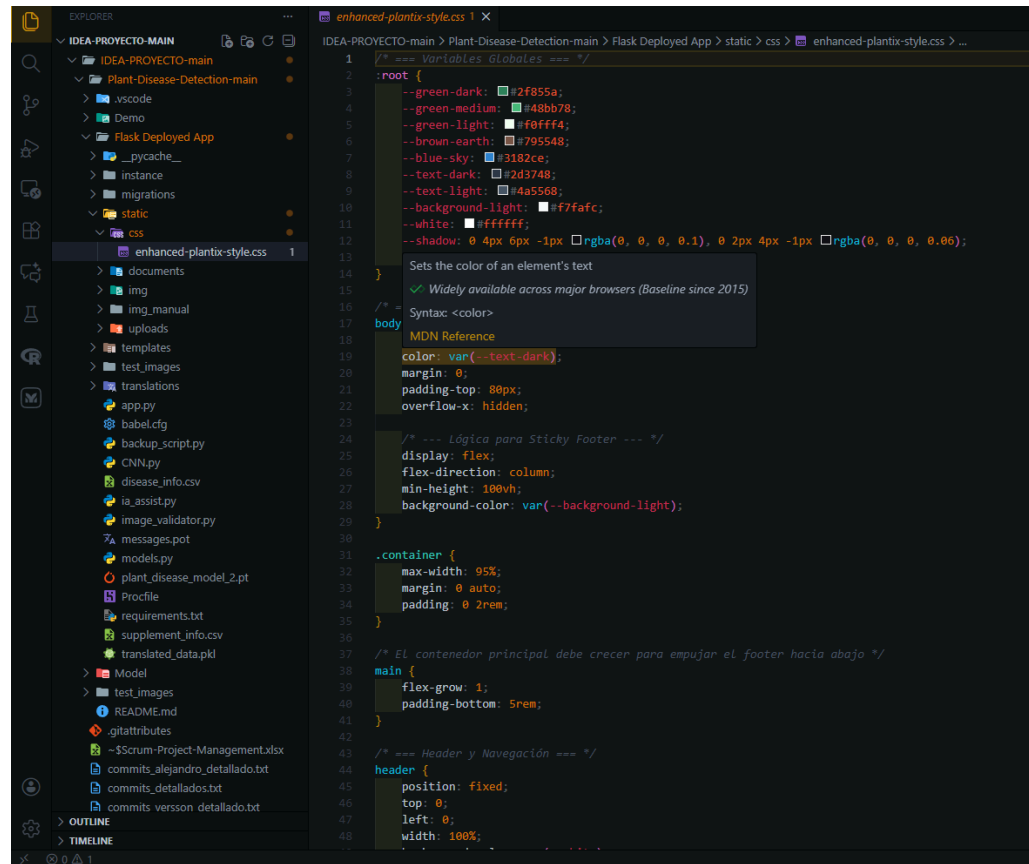


The screenshot shows the PyCharm IDE interface. On the left, the Explorer pane displays the project structure for 'IDEA-PROYECTO-MAIN'. The 'migrations' folder is expanded, showing 'env.py' as the active file. The main editor pane displays the code for 'env.py', which is a Flask migration script. The code includes imports for logging, fileConfig, current_app, and alembic context. It sets up logging and defines functions for getting the database engine and its URL. The code is as follows:

```
1 import logging
2 from logging.config import fileConfig
3
4 from flask import current_app
5
6 from alembic import context
7
8 # this is the Alembic Config object, which provides
9 # access to the values within the .ini file in use.
10 config = context.config
11
12 # Interpret the config file for Python logging.
13 # This line sets up loggers basically.
14 fileConfig(config.config_file_name)
15 logger = logging.getLogger('alembic.env')
16
17
18 def get_engine():
19     try:
20         # this works with Flask-SQLAlchemy<3 and Alchemical
21         return current_app.extensions['migrate'].db.get_engine()
22     except (TypeError, AttributeError):
23         # this works with Flask-SQLAlchemy>=3
24         return current_app.extensions['migrate'].db.engine
25
26
27 def get_engine_url():
28     try:
29         return get_engine().url.render_as_string(hide_password=False).replace(
30             '%', '%%')
31     except AttributeError:
32         return str(get_engine().url).replace('%', '%%')
33
34
35 # add your model's MetaData object here
36 # for 'autogenerate' support
37 # from myapp import mymodel
38 # target_metadata = mymodel.Base.metadata
39 config.set_main_option('sqlalchemy.url', get_engine_url())
40 target_db = current_app.extensions['migrate'].db
41
42 # other values from the config, defined by the needs of env.py,
43 # can be acquired:
44 # my_important_option = config.get_main_option("my_important_option")
45 # ... etc.
46
47
48 def get_metadata():
```



```
1 <!-- ===== Plantas ===== -->
2 <!-- Plantas - Página del Motor de IA -->
3
4 <!-- Copyright (c) 2025 Yerson Calderon Romero. Todos Los derechos reservados. -->
5 <!-- Autor: Yerson Calderon Romero -->
6 <!-- ===== -->
7
8 <!-- -->
9
10 <!-- extends 'base.html' % -->
11 <!-- block title %({{ _('Detector de Enfermedades') }}) - Plantas({{ endblock % -->
12
13 <!-- block content % -->
14
15 <div class="detector-section">
16   <div class="container">
17     <h1 class="section-title">{{ _('Detector de Enfermedades en Plantas Andinas') }}</h1>
18     <p class="section-description">
19       {{ _('Sube una imagen de tu cultivo y nuestro sistema de inteligencia artificial la analizará para detectar posibles enfermedades.') }}
20     </p>
21
22     <!-- Contenedor para mostrar mensajes flash -->
23     <div class="flash-messages">
24       <div class="flash-message" {{ category }}>{{ message }}</div>
25     </div>
26
27     <div class="detector-card">
28       <form action="{{ url_for('ai_engine_page') }}" method="POST" enctype="multipart/form-data">
29         <div class="upload-area" id="uploadArea">
30           <div class="fas fa-cloud-upload-alt upload-icon"></div>
31           <p class="upload-text">{{ _('Arrastra y suelta una imagen aquí o haz clic para seleccionar') }}</p>
32           <input type="file" id="image-input" name="image" accept="image/*" required multiple>
33           <label for="image-input" class="upload-button">{{ _('Seleccionar Imagen') }}</label>
34           <p class="file-name" id="file-name">{{ _('Ningún archivo seleccionado') }}</p>
35         </div>
36
37         <button type="submit" class="analyze-button">
38           <div class="fas fa-search"></div> {{ _('Analizar Imagen') }}
39         </button>
40       </form>
41     </div>
42
43     <div class="instructions">
44       <h2 class="instructions-title">{{ _('Instrucciones') }}</h2>
45       <ul class="instructions-list">
46         <li><div class="fas fa-check-circle"></div> {{ _('Asegúrate de que la imagen sea clara y esté bien iluminada') }}</li>
47       </ul>
48     </div>
49   </div>
50 </div>
```



```
1 /* === Variables Globales === */
2
3 :root {
4   --green-dark: #2f855a;
5   --green-medium: #48bb78;
6   --green-light: #f0fff4;
7   --brown-earth: #795548;
8   --blue-sky: #3182ce;
9   --text-dark: #2d3748;
10   --text-light: #4a5568;
11   --background-light: #f7fafc;
12   --white: #ffffff;
13   --shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1), 0 2px 4px -1px rgba(0, 0, 0, 0.06);
14 }
15
16 /* --- Sets the color of an element's text --- */
17 /* --- Widely available across major browsers (Baseline since 2015) --- */
18 Syntax: <color>
19 MDN Reference
20 color: var(--text-dark);
21 margin: 0;
22 padding-top: 80px;
23 overflow-x: hidden;
24
25 /* --- Lógica para Sticky Footer --- */
26 display: flex;
27 flex-direction: column;
28 min-height: 100vh;
29 background-color: var(--background-light);
30
31 .container {
32   max-width: 95%;
33   margin: 0 auto;
34   padding: 0 2rem;
35 }
36
37 /* El contenedor principal debe crecer para empujar el footer hacia abajo */
38 main {
39   flex-grow: 1;
40   padding-bottom: 5rem;
41 }
42
43 /* --- Header y Navegación --- */
44 header {
45   position: fixed;
46   top: 0;
47   left: 0;
48   width: 100%;
```

6. Diseño de Ingeniería
 - 6.1. Listado de Requerimientos funcionales

PMV1–HU1–Configuración del entorno
Instalar Python y librerías necesarias, verificar compatibilidad con GPU, configurar entorno virtual y crear el repositorio en GitHub.
Criterios de Aceptación
• Python y librerías instaladas correctamente. • GPU reconocida por PyTorch. • Entorno virtual funcional. • Repositorio creado y sincronizado en GitHub.

PMV1-HU2 – Recolección y organización del dataset
Búsqueda, selección, clasificación y documentación del dataset adecuado para entrenar el modelo.
Criterios de Aceptación
• Dataset público encontrado. • Cultivos andinos definidos. • Imágenes clasificadas correctamente. • Fuentes de datos documentadas.

PMV1–HU3–Preprocesamiento de imágenes
Aplicar redimensionamiento, normalización de píxeles, técnicas de aumento de datos y generación del dataset limpio final.
Criterios de Aceptación
• Todas las imágenes normalizadas. • Aumentos aplicados correctamente (flip, gamma, ruido, rotación). • Dataset limpio almacenado sin duplicados. • Scripts reproducibles y documentados.

PMV1–HU4–Diseño de arquitectura base CNN
Definir las capas fundamentales de la CNN, configurar la función de pérdida y el optimizador, realizar el entrenamiento inicial y evaluar su rendimiento preliminar.
Criterios de Aceptación

- Arquitectura definida sin errores.
- Entrenamiento inicial ejecutado correctamente.
- Función de pérdida y optimizador funcionales en PyTorch.
- Métricas preliminares generadas.

PMV1–HU5–Clasificación básica

Entrenar el modelo con dos clases base, validar resultados, mostrar las métricas iniciales de exactitud y probar predicciones sencillas.

Criterios de Aceptación

- Modelo entrenado con dos clases sin fallos.
- Resultados validados con subconjunto del dataset.
- Exactitud calculada correctamente.
- Pruebas de predicción mostradas.

PMV1–HU8–Mostrar nivel de confianza

Calcular probabilidades usando softmax, mostrar el porcentaje de confianza en pantalla, validar casos de baja confianza e integrarlo con el resultado del modelo.

Criterios de Aceptación

- Cálculo de softmax implementado
- Confianza mostrada en la interfaz o salida del modelo.
- Casos de baja confianza identificados.
- Integración completa con la predicción.

PMV2–HU6 – Detección de cultivo

Implementar modelo multiclase para identificar automáticamente el tipo de cultivo.

Criterios de Aceptación

- Clasificación multiclase funcional.
- CNN entrenada para cultivos.
- Precisión validada por cultivo
- Pesos entrenados guardados.

PMV2-HU7 – Diagnóstico por enfermedad
Entrenar un modelo con enfermedades específicas y generar la primera matriz de confusión.
Criterios de Aceptación
• Clases de enfermedades ampliadas. • Modelo entrenado con dataset etiquetado. • Precisión por enfermedad validada. • Matriz de confusión generada.

PMV2-HU9 – Interfaz web
Diseñar la interfaz principal, integrar el formulario para subir imágenes y mostrar los resultados.
Criterios de Aceptación
• Interfaz creada con HTML/CSS/Flask/Django. • Formulario de subida activo. • Backend enlazado con el modelo. • Resultados visibles en la web.

PMV2-HU10 – Subida intuitiva
Habilitar drag-and-drop, previsualización y validación de formato de imagen.
Criterios de Aceptación
• Drag-and-drop funcional. • Subida desde archivos habilitada. • Formatos jpg/png validados. • Vista previa visible.

PMV2-HU15 – Ejemplos visuales
Mostrar imágenes de referencia de hojas sanas y enfermas junto al diagnóstico.
Criterios de Aceptación

- Imágenes de referencia cargadas.
- Visualización junto al diagnóstico.
- Dataset validado como referencia.
- Buena legibilidad en la interfaz.

PMV2-HU16 – Recomendaciones básicas

Asociar recomendaciones a enfermedades detectadas y mostrarlas al usuario.

Criterios de Aceptación

- Recomendaciones asignadas por enfermedad.
- Consejos visibles en la interfaz.
- Coherencia con diagnóstico previo.
- Base inicial creada.

PMV3-HU12 – Transfer Learning

Seleccionar un modelo preentrenado y ajustarlo a las 39 categorías del dataset.

Criterios de Aceptación

- Modelo preentrenado seleccionado.
- Capas adaptadas correctamente.
- Entrenamiento completado.
- Validación superada.

PMV3-HU13 – Métricas de validación

Implementar precisión, recall, F1-score y la matriz de confusión.

Criterios de Aceptación

- Precisión calculada.
- Recall y F1-score implementados.
- Matriz de confusión generada.
- Resultados documentados.

PMV3-HU23 – Visualización de métricas
Generar gráficos de rendimiento y añadirlos al reporte técnico o notebook.
Criterios de Aceptación
• Gráficos de matriz de confusión. • Gráfica de precisión/pérdida por época. • Curva ROC generada. • Integración en reporte o notebook.

PMV3-HU25 – Optimización para Web
Reducir tamaño del modelo y exportarlo para ejecución eficiente en entorno web.
Criterios de Aceptación
• Modelo reducido o podado. • Exportado a TorchScript/ONNX. • Pruebas de inferencia exitosas. • Tiempos de respuesta optimizados.

PMV3-HU27 – Validación externa
Evaluar el modelo con datos externos y comparar resultados.
Criterios de Aceptación
• Dataset externo cargado. • Pruebas ejecutadas con el modelo. • Comparación generada. • Resultados documentados.

PMV3-HU11 – Dataset ampliado
Entrenar nuevamente el modelo con aumentación avanzada y evaluar su generalización.
Criterios de Aceptación
• Aumentación aplicada.

- Reentrenamiento completado.
- Generalización evaluada.
- Comparación con modelo base.

PMV3-HU14-Historial de Diagnósticos

La interfaz web permitirá a los usuarios visualizar un historial de diagnósticos anteriores, mostrando imágenes, fechas y enfermedades detectadas, con opciones de filtrado.

Criterios de Aceptación

- Debe mostrar todos los diagnósticos guardados.
- Permitir filtrar por fecha o por tipo de enfermedad.
- Cada diagnóstico debe incluir imagen, fecha y resultado.
- Debe actualizarse automáticamente al generar un nuevo diagnóstico.

PMV3-HU17-Notas en Diagnóstico

El sistema permitirá agregar notas personalizadas a cada diagnóstico generado, facilitando el registro de observaciones por parte del agricultor.

Criterios de Aceptación

- Las notas deben almacenarse junto al diagnóstico.
- Deben mostrarse dentro del historial.
- El usuario debe poder editar o eliminar notas.
- Las notas deben registrarse sin afectar el resultado del diagnóstico.

PMV3-HU18-Validación de Calidad de Imagen

La aplicación evaluará automáticamente la calidad de la imagen subida para detectar si está borrosa, oscura o incompleta antes del análisis.

Criterios de Aceptación

- Identificar imágenes borrosas o con bajo contraste.
- Mostrar advertencia cuando la calidad no sea suficiente.
- Bloquear el diagnóstico si la imagen no cumple estándares mínimos.
- Registrar estadísticas de errores de calidad.

PMV3-HU24-Múltiples Imágenes por Cultivo
El sistema permitirá analizar varias imágenes de un mismo cultivo para obtener un diagnóstico más preciso basado en evidencias múltiples.
Criterios de Aceptación
● Permite subir de 2 a 5 imágenes simultáneamente. ● Analizar cada imagen individualmente. ● Generar un diagnóstico consolidado a partir de las imágenes. ● Mostrar promedio del porcentaje de confianza.

PMV3-HU26-Alertas Tempranas
La interfaz web mostrará alertas cuando se detecten patrones de brotes de enfermedades en cultivos analizados recientemente.
Criterios de Aceptación
● Identificar la recurrencia elevada de una enfermedad. ● Generar alerta visual dentro del sistema. ● Mostrar resumen estadístico del brote. ● Actualizar las alertas en tiempo real.

PMV3-HU19-Clasificación de Severidad
El modelo clasificará el nivel de severidad de la enfermedad como leve, moderada o grave, según los signos visibles en la hoja.
Criterios de Aceptación
● El modelo debe devolver uno de los tres niveles de severidad. ● Debe mostrarse junto al diagnóstico principal. ● La clasificación debe basarse en características visuales reales. ● Debe reflejarse en los reportes del usuario.

PMV3-HU22-Retroalimentación del Usuario
--

El sistema incluirá un formulario para que el usuario evalúe la precisión del diagnóstico y brinde comentarios sobre la utilidad de la herramienta.

Criterios de Aceptación

- Debe permitir enviar una puntuación del 1 al 5.
- Guardar la retroalimentación en la base de datos.
- Mostrar estadísticas globales de satisfacción.
- No debe afectar el rendimiento del sistema.

PMV3-HU28-Backup Automático

El sistema realizará copias de seguridad automáticas de los diagnósticos e imágenes, asegurando la integridad y disponibilidad de los datos.

Criterios de Aceptación

- Programar backups automáticos según intervalo definido.
- Guardar copias en un servicio seguro externo.
- Permitir restaurar un respaldo cuando sea necesario.
- Registrar fecha y hora de cada respaldo generado.

PMV3-HU20-Exportación de Datos Anonimizados

El sistema permitirá exportar los diagnósticos en CSV o Excel, eliminando cualquier información personal asociada al usuario.

Criterios de Aceptación

- Exportar datos solo con información técnica del diagnóstico.
- Verificar anonimización de nombres o ubicaciones.
- Archivo debe ser compatible con Excel/Sheets.
- Debe permitir elegir rango de fechas.

PMV3-HU21-Soporte Multilenguaje

La interfaz web ofrecerá soporte en español y quechua, permitiendo al usuario cambiar el idioma del sistema desde la configuración.

Criterios de Aceptación

- Archivos de traducción creados correctamente.
- Todos los textos visibles deben traducirse.
- El cambio de idioma debe aplicarse en tiempo real.
- Persistencia del idioma entre sesiones.

PMV3-HU30-Manual de Usuario

Se desarrollará un manual completo que explique el funcionamiento del sistema, incluyendo ejemplos de uso y capturas de pantalla.

Criterios de Aceptación

- Manual disponible en PDF.
- Debe incluir pasos detallados y capturas reales.
- Secciones organizadas por funcionalidades del sistema.
- Versión accesible desde la interfaz web.

PMV3-HU29-Documentación Técnica

Se elaborará la documentación técnica del proyecto, describiendo la arquitectura del sistema, flujo de datos, dependencias y resultados finales.

Criterios de Aceptación

- Debe incluir arquitectura general y diagramas.
- Explicar la configuración del modelo CNN.
- Describir pipeline de entrenamiento e inferencia.
- Entregar versión final para repositorio.

6.2. Estimación de tiempos

1. Estimación inicial:

- Estimación total (suma de todas las historias de usuario): **256 horas**.
- Este total representa el esfuerzo combinado del equipo, ya distribuido entre las 30 historias.

2. Capacidad del Equipo por Semana:

Los integrantes indicaron su disponibilidad diaria:

- **Alejandro:** 2.5 horas/día
- **André:** 2.0 horas/día
- **Yersson:** 4.0 horas/día

Promedio por persona (día):

$$\text{Promedio} = \frac{2.5 + 2.0 + 4.0}{3} = \frac{8.5}{3} = 2.83 \text{ horas/día (aprox.)}$$

(Se redondea a **2.83 h/día por persona** — este será el valor de referencia de disponibilidad diaria).

Capacidad semanal por persona (7 días):

Capacidad total del equipo por semana (3 integrantes):

3. **Verificación para ≤ 10 Semanas:**

- Total, de horas requeridas: 256 h
- Capacidad total del equipo en 10 semanas:

Resultado: La capacidad disponible (595 h) es ampliamente superior a las 256 h requeridas. El equipo tiene **holgura** para completar el proyecto dentro de 10 semanas y absorber imprevistos, pruebas adicionales o correcciones.

4. **Distribución del Trabajo:**

Horas por semana requeridas (equipo):

Horas por persona por semana:

Horas por persona por día (7 días):

6.3. **Estimación de épicas**

Épica 1 – Preparación y Configuración Técnica

Descripción: Configuración del entorno de desarrollo, dataset y arquitectura inicial del modelo CNN en PyTorch.

- HU-1: Configuración del entorno (8h, Prioridad: Alta)
- HU-2: Recolección y organización del dataset (12h, Prioridad: Alta)

- HU-3: Preprocesamiento de imágenes (8h, Prioridad: Alta)
- HU-4: Diseño de arquitectura base CNN (12h, Prioridad: Alta)

Total, estimado Épica 1: 40h

Épica 2 – Funcionalidades Principales (Diagnóstico Web)

Descripción: Funcionalidades base para diagnóstico de hojas mediante CNN, interfaz web y experiencia mínima viable.

- HU-5: Clasificación básica (8h, Prioridad: Alta)
- HU-6: Detección de cultivo (12h, Prioridad: Alta)
- HU-7: Diagnóstico específico por enfermedad (16h, Prioridad: Alta)
- HU-8: Nivel de confianza (4h, Prioridad: Alta)
- HU-9: Interfaz web (12h, Prioridad: Alta)
- HU-10: Subida intuitiva de imágenes (4h, Prioridad: Alta)

Total, estimado Épica 2: 56h

Épica 3 – Optimización y Validación del Modelo

Descripción: Mejoras de rendimiento y precisión del modelo usando técnicas avanzadas.

- HU-11: Entrenamiento con dataset ampliado (8h, Prioridad: Media)
- HU-12: Transfer learning con modelos preentrenados (16h, Prioridad: Alta)
- HU-13: Métricas de validación (8h, Prioridad: Alta)
- HU-23: Gráficos de métricas (8h, Prioridad: Alta)
- HU-25: Optimización del modelo para despliegue web (12h, Prioridad: Alta)
- HU-27: Validación con dataset externo (8h, Prioridad: Alta)

Total, estimado Épica 3: 60h

Épica 4 – Funcionalidades de Usuario y Experiencia

Descripción: Funcionalidades adicionales para enriquecer la experiencia del agricultor en el uso de la aplicación web.

- HU-14: Historial de diagnósticos (8h, Prioridad: Media)
- HU-15: Ejemplos visuales (4h, Prioridad: Media)
- HU-16: Recomendaciones básicas (8h, Prioridad: Alta)
- HU-17: Clasificación de severidad (8h, Prioridad: Media)
- HU-18: Validación de calidad de imagen (8h, Prioridad: Media)
- HU-24: Guardar notas junto al diagnóstico (4h, Prioridad: Media)
- HU-26: Alertas tempranas en interfaz (8h, Prioridad: Media)

Total, estimado Épica 4: 48h

Épica 5 – Reportes, Exportaciones e Inclusión

Descripción: Generación de reportes, exportación de datos, idiomas y soporte para la toma de decisiones.

- HU-19: Reportes PDF (8h, Prioridad: Media)
- HU-20: Exportación de datos anonimizados (8h, Prioridad: Media)
- HU-21: Multilenguaje Español/Quechua (8h, Prioridad: Media)
- HU-22: Retroalimentación del usuario (4h, Prioridad: Media)
- HU-30: Manual de usuario (8h, Prioridad: Media)

Total, estimado Épica 5: 36h

Épica 6 – Soporte, Seguridad y Documentación

Descripción: Mantenimiento del sistema, seguridad de datos y documentación del proyecto.

- HU-28: Backup automático del modelo (4h, Prioridad: Media)
- HU-29: Documentación técnica (12h, Prioridad: Alta)

Total, estimado Épica 6: 16h

Resumen de horas por épica

- **Épica 1 – Preparación y Configuración Técnica: 40h**
- **Épica 2 – Funcionalidades Principales (Diagnóstico Web): 56h**
- **Épica 3 – Optimización y Validación del Modelo: 60h**
- **Épica 4 – Funcionalidades de Usuario y Experiencia: 48h**
- **Épica 5 – Reportes, Exportaciones e Inclusión: 36h**
- **Épica 6 – Soporte, Seguridad y Documentación: 16h**

Total, general: 256h.

6.4. **Product backlog según prioridades y estimaciones**

Ordenado de mayor a menor prioridad.

Épicas de Prioridad Alta:

Épica 2 – Funcionalidades Principales (Diagnóstico Web) (56h, Prioridad: Alta)

Incluye las funcionalidades esenciales para el diagnóstico en la interfaz web.

- HU-5: Clasificación básica (8h, Alta)
- HU-6: Detección de cultivo (12h, Alta)
- HU-7: Diagnóstico específico por enfermedad (16h, Alta)
- HU-8: Nivel de confianza (4h, Alta)
- HU-9: Interfaz web (12h, Alta)
- HU-10: Subida intuitiva de imágenes (4h, Alta)

Épica 3 – Optimización y Validación del Modelo (60h, Prioridad: Alta)

Mejoras de rendimiento y validación del modelo CNN en PyTorch.

- HU-11: Entrenamiento con dataset ampliado (8h, Media)
- HU-12: Transfer learning (16h, Alta)
- HU-13: Métricas de validación (8h, Alta)
- HU-23: Gráficos de métricas (8h, Alta)
- HU-25: Optimización del modelo para despliegue web (12h, Alta)

- HU-27: Validación con dataset externo (8h, Alta)

Épica 1 – Preparación y Configuración Técnica (40h, Prioridad: Alta)

Etapas iniciales para preparar el entorno y la arquitectura base.

- HU-1: Configuración del entorno (8h, Alta)
- HU-2: Recolección y organización del dataset (12h, Alta)
- HU-3: Preprocesamiento de imágenes (8h, Alta)
- HU-4: Diseño de arquitectura base CNN (12h, Alta)

Épicas de Prioridad Media:

Épica 4 – Funcionalidades de Usuario y Experiencia (48h, Prioridad: Media)

Aporta valor agregado para la interacción y usabilidad del sistema.

- HU-14: Historial de diagnósticos (8h, Media)
- HU-15: Ejemplos visuales (4h, Media)
- HU-16: Recomendaciones básicas (8h, Alta)
- HU-17: Clasificación de severidad (8h, Media)
- HU-18: Validación de calidad de imagen (8h, Media)
- HU-24: Guardar notas junto al diagnóstico (4h, Media)
- HU-26: Alertas tempranas en interfaz (8h, Media)

Épica 5 – Reportes, Exportaciones e Inclusión (36h, Prioridad: Media)

Facilita el análisis y la inclusión de agricultores quechua-hablantes.

- HU-19: Reportes PDF (8h, Media)
- HU-20: Exportación de datos anonimizados (8h, Media)
- HU-21: Multilenguaje Español/Quechua (8h, Media)
- HU-22: Retroalimentación del usuario (4h, Media)
- HU-30: Manual de usuario (8h, Media)

Épica de Prioridad Baja:

Épica 6 – Soporte, Seguridad y Documentación (16h, Prioridad: Baja)

Garantiza la seguridad y documentación del sistema.

- HU-28: Backup automático del modelo (4h, Media)
- HU-29: Documentación técnica (12h, Alta)

6.5. Definición de Sprints

Capacidad real del sprint (equipo)

- Promedio diario por persona: 2.83 h/día
- Días del sprint: 7 días
- Horas disponibles por persona/sprint:
- Equipo de 3 personas:

Cada sprint tiene una capacidad de ~59 horas.

Total de esfuerzo a cubrir

- Total del backlog: 256 h
- Número de sprints necesarios:

Se necesitarán 5 sprints (los últimos con menos carga).

Asignación de historias de usuario a cada sprint

Agruparemos HU en orden de prioridad (alta → media → baja) hasta llenar ~59h por sprint.

Sprint 1 (59h máx – Total: 56h)

Enfoque: Preparación técnica y base del sistema.

- HU-1: Configuración del entorno (8h)
- HU-2: Recolección y organización del dataset (12h)
- HU-3: Preprocesamiento de imágenes (8h)
- HU-4: Diseño de arquitectura base CNN (12h)

- HU-5: Clasificación básica (8h)
- HU-8: Nivel de confianza (4h)

Total Sprint 1 = 56h

Sprint 2 (59h máx – Total: 56h)

Enfoque: Funcionalidades principales en la interfaz web.

- HU-6: Detección de cultivo (12h)
- HU-7: Diagnóstico específico por enfermedad (16h)
- HU-9: Interfaz web (12h)
- HU-10: Subida intuitiva de imágenes (4h)
- HU-16: Recomendaciones básicas (8h)
- HU-15: Ejemplos visuales (4h)

Total Sprint 2 = 56h

Sprint 3 (59h máx – Total: 60h)

Enfoque: Optimización y validación del modelo.

- HU-12: Transfer learning (16h)
- HU-13: Métricas de validación (8h)
- HU-23: Gráficos de métricas (8h)
- HU-25: Optimización del modelo para web (12h)
- HU-27: Validación con dataset externo (8h)
- HU-11: Dataset ampliado (8h)

Total Sprint 3 = 60h

Sprint 4 (59h máx – Total: 52h)

Enfoque: Funcionalidades de usuario y reportes básicos.

- HU-14: Historial de diagnósticos (8h)
- HU-17: Clasificación de severidad (8h)
- HU-18: Validación de calidad de imagen (8h)

- HU-24: Notas en diagnóstico (4h)
 - HU-26: Alertas tempranas (8h)
 - HU-19: Reportes PDF (8h)
 - HU-22: Retroalimentación (4h)
 - HU-28: Backup automático del modelo (4h)
- Total Sprint 4 = 52h**

Sprint 5 (59h máx – Total: 32h)

Enfoque: Inclusión, documentación y cierre del proyecto.

- HU-20: Exportación de datos anonimizados (8h)
 - HU-21: Multilenguaje (8h)
 - HU-30: Manual de usuario (8h)
 - HU-29: Documentación técnica (12h)
- Total Sprint 5 = 32h**

Tabla de los Sprint:

<i>Sprint</i>	<i>Historias de Usuario</i>	<i>Actividades</i>	<i>Horas</i>	<i>Total Sprint</i>
<i>Sprint 1</i>	HU-1, HU-2, HU-3, HU-4, HU-5, HU-8	Configuración del entorno; Recolección y organización del dataset; Preprocesamiento de imágenes; Diseño de CNN base; Clasificación básica; Mostrar nivel de confianza	8 + 12 + 8 + 12 + 8 + 4	56h
<i>Sprint 2</i>	HU-6, HU-7, HU-9, HU-10, HU-16, HU-15	Detección de cultivo; Diagnóstico específico por enfermedad; Interfaz web; Subida intuitiva de imágenes; Recomendaciones básicas; Ejemplos visuales	12 + 16 + 12 + 4 + 8 + 4	56h
<i>Sprint 3</i>	HU-12, HU-13, HU-23, HU-25, HU-27, HU-11	Transfer learning; Generar métricas; Visualización de métricas; Optimización del modelo para web; Validación con dataset externo; Entrenamiento con dataset ampliado	16 + 8 + 8 + 12 + 8 + 8	60h
<i>Sprint 4</i>	HU-14, HU-17, HU-18, HU-24, HU-26, HU-19, HU-22, HU-28	Historial de diagnósticos; Clasificación de severidad; Validación de calidad de imagen; Notas en diagnóstico; Alertas tempranas; Reportes PDF; Retroalimentación; Backup automático	8 + 8 + 8 + 4 + 8 + 8 + 4 + 4	52h

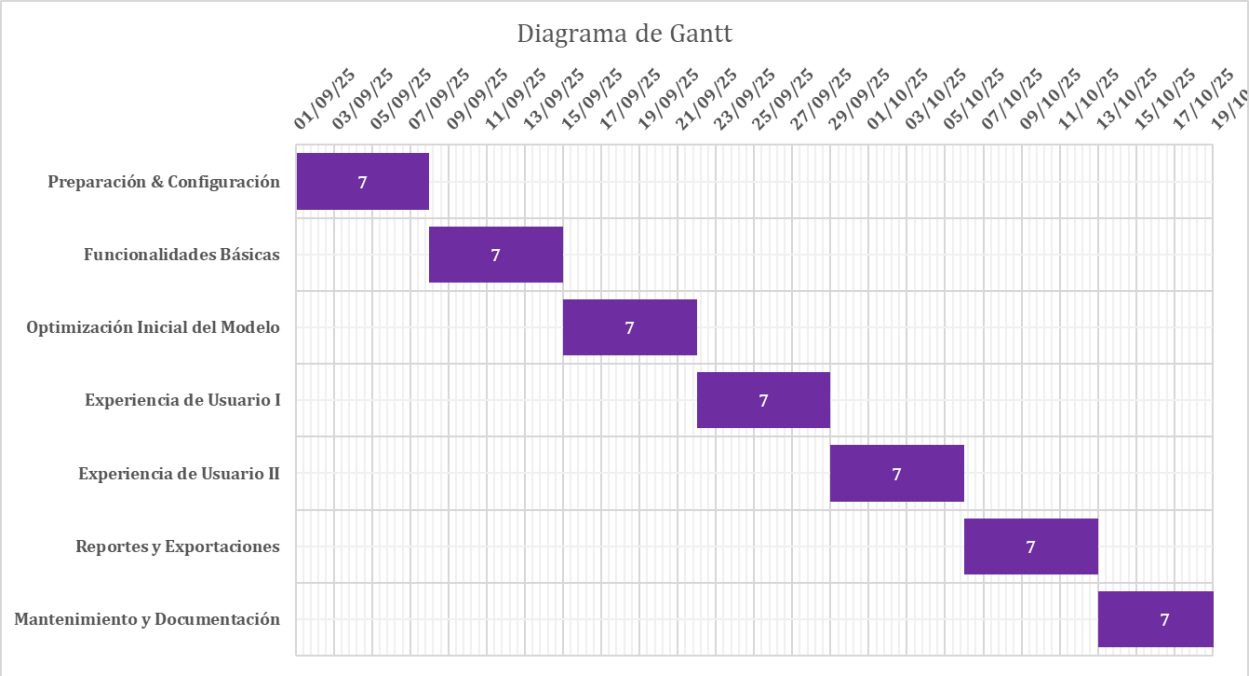
<i>Sprint 5</i>	HU-20, HU-21, HU-30, HU-29	Exportación de datos anonimizados; Multilenguaje; Manual de usuario; Documentación técnica	8 + 8 + 8 + 12	32h
------------------------	----------------------------	--	----------------	-----

7. VII. GESTIÓN DEL PROYECTO:

7.1. Diagrama de Gantt (Línea Base)

<i>Nombre Actividad</i>	<i>Actividades</i>	<i>Inicio</i>	<i>Días</i>	<i>Final</i>	<i>Épicas/HU</i>
<i>Preparación & Configuración</i>	Configuración del entorno; Recolección y organización del dataset; Preprocesamiento de imágenes; Diseño de la CNN base	01/09/2025	7	07/09/2025	Épica 1 (HU-1 a HU-4)
<i>Funcionalidades Básicas</i>	Clasificación sana/enferma; Detección de cultivo; Diagnóstico específico; Nivel de confianza; Interfaz web; Subida de imágenes	08/09/2025	7	14/09/2025	Épica 2 (HU-5 a HU-10)
<i>Optimización Inicial del Modelo</i>	Entrenamiento con dataset ampliado; Transfer learning; Métricas de validación; Gráficos de métricas; Optimización para web; Validación externa	15/09/2025	7	21/09/2025	Épica 3 (HU-11 a HU-13, HU-23, HU-25, HU-27)
<i>Experiencia de Usuario I</i>	Historial de diagnósticos; Ejemplos visuales; Recomendaciones básicas	22/09/2025	7	28/09/2025	Épica 4 (HU-14, HU-15, HU-16)
<i>Experiencia de Usuario II</i>	Clasificación de severidad; Validación de calidad de imagen; Notas en diagnósticos; Alertas tempranas	29/09/2025	7	05/10/2025	Épica 4 (HU-17, HU-18, HU-24, HU-26)
<i>Reportes y Exportaciones</i>	Reportes PDF; Exportación de datos anonimizados; Soporte multilenguaje; Retroalimentación; Manual de usuario	06/10/2025	7	12/10/2025	Épica 5 (HU-19 a HU-22, HU-30)
<i>Mantenimiento y Documentación</i>	Backup automático del modelo; Documentación técnica	13/10/2025	7	19/10/2025	Épica 6 (HU-28, HU-29)

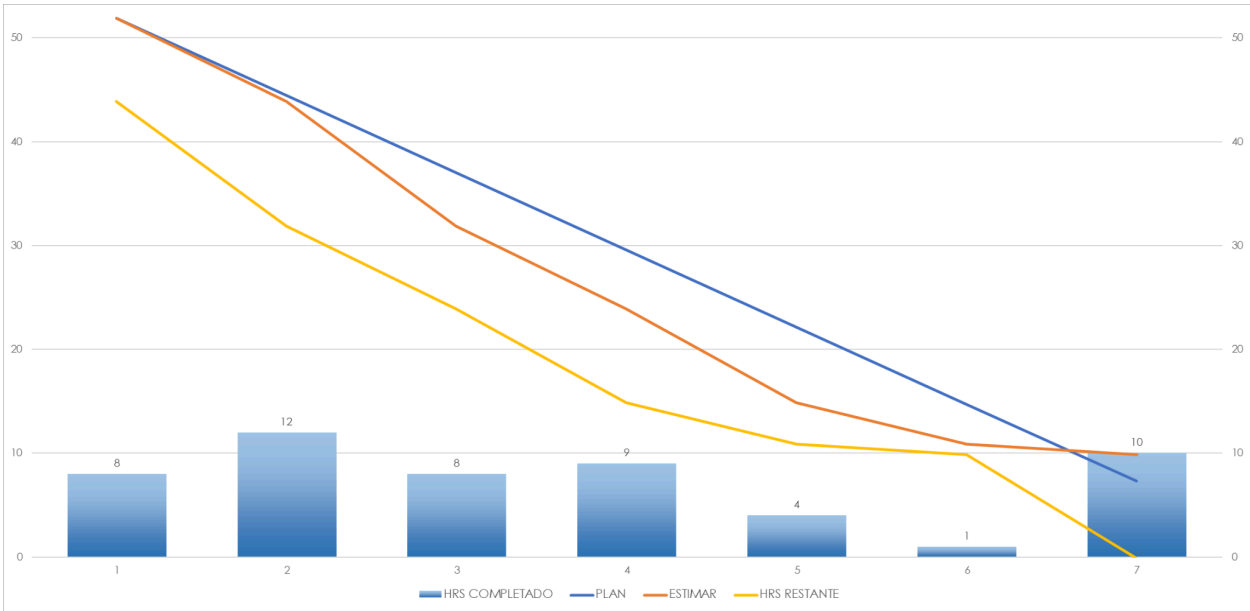
Diagrama de Gantt:



7.2. Diagrama de Gantt(Ejecutado)
Sprint 1.

ESTRUCTURA DE DESGLOSE DEL	TÍTULO DE LA TAREA	PROPIETARIO DE LA TAREA	CANTIDAD DE TRABAJO EN			CARRERA	FECHA DE INICIO	FECHA DE VENCIMIENTO	DURACIÓN	PCT DE TAREA COMPLETADA
			ESTIMAR	HORAS COMPLETADO	RESTANTE					
1	HU-1: Configuración del entorno (8h)		8	8	0		13/09/2025	19/09/2025	7	100%
1.1	Instalar Python y librerías necesarias	Alejandro	4	4	0					100%
1.2	Verificar compatibilidad con GPU.	Alejandro	2	2	0					100%
1.3	Configurar entorno virtual.	Alejandro	1	1	0					100%
1.4	Crear repositorio en GitHub.	Alejandro	1	1	0					100%
2	HU-2: Recolección y organización del dataset (12h)		12	12	0		13/09/2025	19/09/2025	7	100%
2.1	Buscar datasets públicos	Yersson	1	1	0					100%
2.2	Seleccionar cultivos andinos	Yersson	2	2	0					100%
2.3	Clasificar imágenes	Yersson	6	6	0					100%
2.4	Documentar la fuente de datos.	Yersson	3	3	0					100%
3	HU-3: Preprocesamiento de imágenes (8h)		8	8	0		13/09/2025	19/09/2025	7	100%
3.1	Redimensionar imágenes	Yersson	3	3	0					100%
3.2	Normalizar valores de píxeles.	Yersson	2	2	0					100%
3.2.1	Implementar técnicas de aumento de datos	Yersson	2	2	0					100%
3.2.2	Guardar dataset limpio.	Yersson	1	1	0					100%
4	HU-4: Diseño de arquitectura base CNN (12h)		12	12	0		13/09/2025	19/09/2025	7	100%
4.1	Definir capas de la red	Andree	2	2	0					100%
4.2	Configurar función de pérdida y optimizador	Andree	2	2	0					100%
4.3	Implementar entrenamiento inicial.	Andree	6	6	0					100%
4.4	Evaluar desempeño preliminar.	Andree	2	2	0					100%
5	HU-5: Clasificación básica (8h)		8	8	0		13/09/2025	19/09/2025	7	100%
5.1	Entrenar modelo con dos clases	Andree	3	3	0					100%
5.2	Validar con un subconjunto del dataset.	Andree	2	2	0					100%
5.3	Mostrar métricas de exactitud.	Andree	2	2	0					100%
5.4	Probar predicciones básicas.	Andree	1	1	0					100%
6	HU-8: Mostrar nivel de confianza (4h)		4	4	0		13/09/2025	19/09/2025	7	100%
6.1	Calcular softmax para probabilidades.	Alejandro	1	1	0					100%
6.2	Mostrar % de confianza en resultados.	Alejandro	1	1	0					100%
6.3	Validar casos de baja confianza.	Alejandro	1	1	0					100%
6.4	Integrar con salida del modelo.	Alejandro	1	1	0					100%
DATOS DE DESGASTE			ESTIMAR	COMPLETADO	RESTANTE	DÍAS	ESTIMAR DÍAS	DÍA		
		TOTAL DE HORAS	52	52	0	7	7.42857			

DÍA	1	2	3	4	5	6	7
PLAN	52	45	37	30	22	15	7
ESTIMAR	52	44	32	24	15	11	10
S COMPLETADO	8	12	8	9	4	1	10
HRS RESTANTE	44	32	24	15	11	10	0

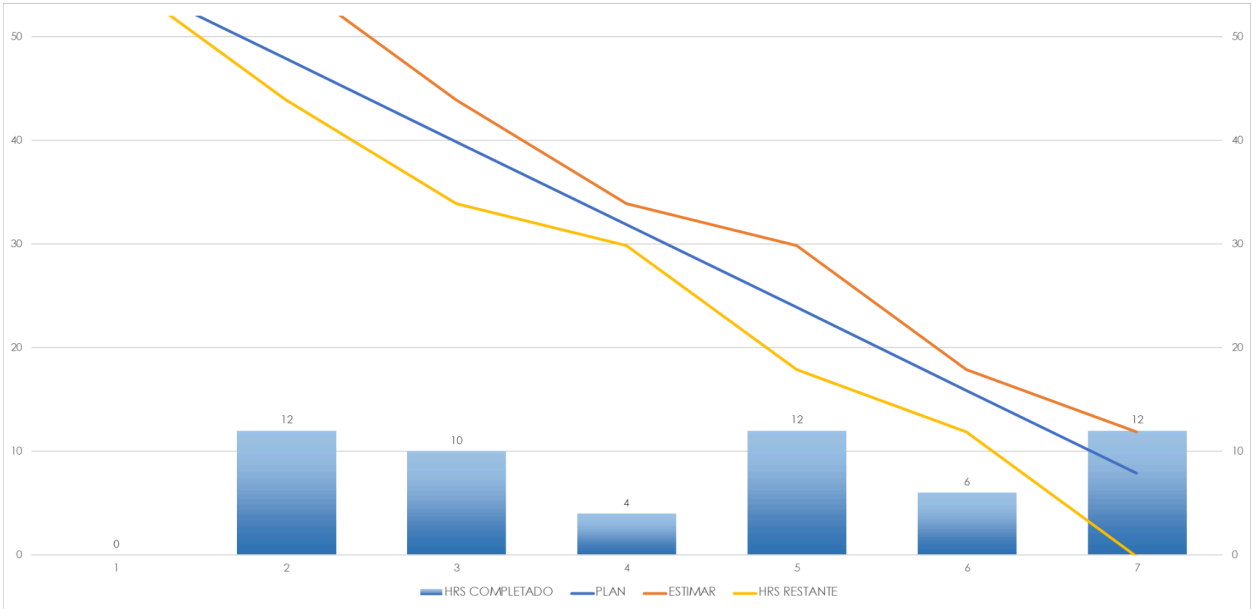


Sprint 2.

ESTRUCTURA DE DESGLOSE DEL	TÍTULO DE LA TAREA	PROPIETARIO DE LA TAREA	CANTIDAD DE TRABAJO EN HORAS			CARRERA	FECHA DE INICIO	FECHA DE VENCIMIENTO	DURACIÓN	PCT DE TAREA COMPLETADA
			ESTIMAR	COMPLETADO	RESTANTE					
1	HU-6: Detección de cultivo (12h)		12	12	0		20/09/2025	26/09/2025	7	100%
1.1	Implementar clasificación multiclase (pa	Yersson	6	6	0					100%
1.2	Entrenar CNN ampliada.	Yersson	2	2	0					100%
1.3	Validar precisión por cultivo.	Yersson	2	2	0					100%
1.4	Guardar pesos entrenados.	Yersson	2	2	0					100%
2	HU-7: Diagnóstico específico por enfermedad (16h)		16	16	0		20/09/2025	26/09/2025	7	100%
2.1	Ampliar clases a enfermedades comunes	Yersson	7	7	0					100%
2.2	Entrenar modelo con dataset etiquetado	Yersson	3	3	0					100%
2.3	Validar métricas de precisión por enferme	Yersson	3	3	0					100%
2.4	Generar matriz de confusión inicial.	Yersson	3	3	0					100%
3	HU-9: Interfaz web (12h)		12	12	0		20/09/2025	26/09/2025	7	100%
3.1	Diseñar frontend básico con HTML/CSS/	Yersson	5	5	0					100%
3.2	Integrar formulario para subir imágenes.	Yersson	2	2	0					100%
3.2.1	Conectar backend con modelo CNN.	Yersson	4	4	0					100%
3.2.2	Mostrar resultados en la web.	Yersson	1	1	0					100%
4	HU-10: Subida intuitiva de imágenes (4h)		4	4	0		20/09/2025	26/09/2025	7	100%
4.1	Implementar arrastrar y soltar.	Yersson	1	1	0					100%
4.2	Permitir subida desde archivos locales.	Yersson	1	1	0					100%
4.3	Validar formato de imágenes (jpg, png).	Yersson	1	1	0					100%
4.4	Mostrar preview de la imagen.	Yersson	1	1	0					100%
5	HU-16: Recomendaciones básicas (8h)		8	8	0		20/09/2025	26/09/2025	7	100%
5.1	Asociar recomendaciones generales a enf	Yersson	3	3	0					100%
5.2	Mostrar consejos simples en interfaz.	Yersson	2	2	0					100%
5.3	Validar con diagnóstico previo.	Yersson	2	2	0					100%
5.4	Crear base inicial de conocimientos.	Yersson	1	1	0					100%

6	HU-15: Ejemplos visuales (4h)		4	4	0		20/09/2025	26/09/2025	7	100%
6.1	Incluir imágenes de hojas sanas/enfermas	Yersson	1	1	0					100%
6.2	Mostrar junto al diagnóstico.	Yersson	1	1	0					100%
6.3	Validar con dataset de referencia.	Yersson	1	1	0					100%
6.4	Asegurar legibilidad en interfaz.	Yersson	1	1	0					100%
DATOS DE DESGASTE			ESTIMAR	COMPLETADO	RESTANTE	DÍAS	EST/DÍAS	DÍA		
TOTAL DE HORAS			56	56	0	7	8.00000			

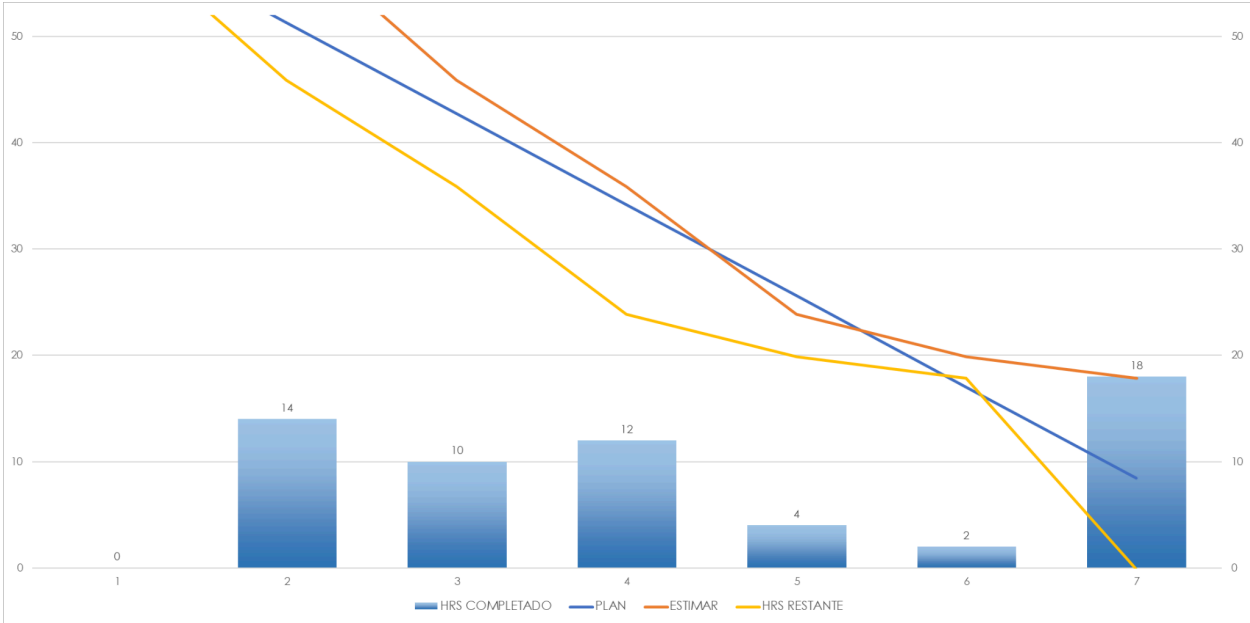
DÍA	1	2	3	4	5	6	7
PLAN	56	48	40	32	24	16	8
ESTIMAR	56	56	44	34	30	18	12
COMPLETADO	0	12	10	4	12	6	12
HRS RESTANTE	56	44	34	30	18	12	0



Sprint 3.

ESTRUCTURA DE DESGLOSE DEL	TÍTULO DE LA TAREA	PROPIETARIO DE LA TAREA	CANTIDAD DE TRABAJO EN HORAS			CARRERA	FECHA DE INICIO	FECHA DE VENCIMIENTO	DURACIÓN	PCT DE TAREA COMPLETADA
			ESTIMAR	COMPLETADO	RESTANTE					
1	HU-12: Transfer Learning (16h)		16	16	0		27/09/2025	03/10/2025	7	100%
1.1	Seleccionar modelo preentrenado (	Yersson	4	4	0					100%
1.2	Adaptar capas finales para 39 categ	Yersson	8	8	0					100%
1.3	Entrenar y validar el modelo ajusta	Yersson	4	4	0					100%
2	HU-13: Métricas de Validación (8h)		8	8	0		27/09/2025	03/10/2025	7	100%
2.1	Implementar cálculo de precisión, r	Yersson	1	1	0					100%
2.2	Generar matriz de confusión con da	Yersson	1	1	0					100%
2.3	Documentar resultados de validació	Yersson	6	6	0					100%
3	HU-23: Visualización de Métricas (8h)		8	8	0		27/09/2025	03/10/2025	7	100%
3.1	Crear gráficos de matriz de confusió	Alejandro	2	2	0					100%
3.2	Generar gráfica de precisión y pérdi	Alejandro	2	2	0					100%
3.3	Integrar visualizaciones en reporte v	Alejandro	4	4	0					100%
4	HU-25: Optimización para Web (12h)		12	12	0		27/09/2025	03/10/2025	7	100%
4.1	Reducir tamaño del modelo usando	Alejandro	3	3	0					100%
4.2	Exportar modelo optimizado a form	Alejandro	3	3	0					100%
4.3	Probar tiempos de inferencia en ent	Alejandro	6	6	0					100%
5	HU-27: Validación con Dataset Externo (8h)		8	8	0		27/09/2025	03/10/2025	7	100%
5.1	Cargar y preparar dataset externo i	Andree	3	3	0					100%
5.2	Ejecutar modelo entrenado sobre d	Andree	4	4	0					100%
5.3	Comparar métricas con dataset ori	Andree	1	1	0					100%
6	HU-11: Entrenamiento con Dataset Ampliado (8h)		8	8	0		27/09/2025	03/10/2025	7	100%
6.1	Incorporar imágenes aumentadas (	Andree	4	4	0					100%
6.2	Reentrenar modelo con dataset am	Andree	3	3	0					100%
6.3	Evaluar mejoras de generalización r	Andree	1	1	0					100%
DATOS DE DESGASTE			ESTIMAR	COMPLETADO	RESTANTE	DÍAS	EST/DÍAS	DÍA		
TOTAL DE HORAS			60	60	0	7	8.57143			

DÍA	1	2	3	4	5	6	7
PLAN	60	51	43	34	26	17	9
ESTIMAR	60	60	46	36	24	20	18
S COMPLETADO	0	14	10	12	4	2	18
HRS RESTANTE	60	46	36	24	20	18	0

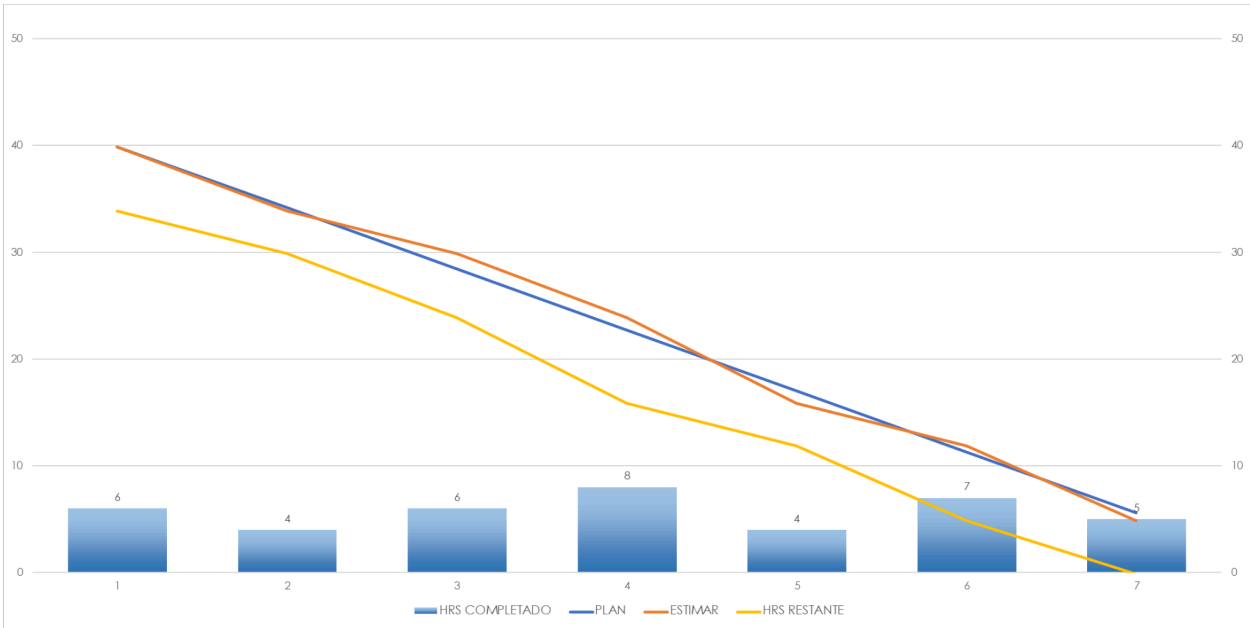


Sprint 4.

ESTRUCTURA DE DESGLOSE DEL	TÍTULO DE LA TAREA	PROPIETARIO DE LA TAREA	CANTIDAD DE TRABAJO EN			CARRERA	FECHA DE INICIO	FECHA DE VENCIMIENTO	DURACIÓN	PCT DE TAREA COMPLETADA
			ESTIMAR	HORAS COMPLETADO	RESTANTE					
1	HU-14: Historial de Diagnósticos (8h)		8	8	0		08/10/2025	14/10/2025	7	100%
1.1	Diseñar estructura de base de datos	Yersson	4	4	0					100%
1.2	Implementar visualización del historial	Yersson	2	2	0					100%
1.3	Permitir filtrado por fecha o tipo de cultivo	Yersson	2	2	0					100%
2	HU-17: Notas en Diagnóstico (4h)		4	4	0		08/10/2025	14/10/2025	7	100%
2.1	Crear campo para añadir notas o comentarios	Yersson	2	2	0					100%
2.2	Guardar notas junto a la imagen y registrar	Yersson	1	1	0					100%
2.3	Mostrar notas en el historial de diagnósticos	Yersson	1	1	0					100%
3	HU-18: Validación de Calidad de Imagen (8h)		8	8	0		08/10/2025	14/10/2025	7	100%
3.1	Implementar detección de imágenes de baja calidad	Yersson	4	4	0					100%
3.2	Notificar al usuario cuando la imagen sea de baja calidad	Yersson	3	3	0					100%
3.3	Registrar estadísticas de calidad de imágenes	Yersson	1	1	0					100%
4	HU-24: Múltiples Imágenes por Cultivo (8h)		8	8	0		08/10/2025	14/10/2025	7	100%
4.1	Permitir carga simultánea de varias imágenes	Alejandro	4	4	0					100%
4.2	Analizar todas las imágenes antes de guardarlas	Alejandro	3	3	0					100%
4.3	Mostrar resultado consolidado con la imagen principal	Alejandro	1	1	0					100%
5	HU-26: Alertas Tempranas (8h)		8	8	0		08/10/2025	14/10/2025	7	100%
5.1	Configurar reglas para detectar patrones de enfermedad	Alejandro	4	4	0					100%
5.2	Generar alertas visuales cuando haya una alerta	Alejandro	3	3	0					100%
5.3	Mostrar resumen de alertas en la interfaz de usuario	Alejandro	1	1	0					100%
6	HU-19: Clasificación de Severidad (8h)		8	8	0		08/10/2025	14/10/2025	7	100%
6.1	Añadir etiquetas de severidad (leve, moderada, grave)	Alejandro	4	4	0					100%
6.2	Ajustar el modelo CNN para salida de severidad	Alejandro	3	3	0					100%
6.3	Mostrar nivel de severidad junto con la imagen	Alejandro	1	1	0					100%

8	HU-28: Backup Automático (4h)		4	4	0		08/10/2025	14/10/2025	7	100%
8.1	Configurar tareas automáticas de r	Andree	2	2	0					100%
8.2	Guardar copias en almacenamiento	Andree	1	1	0					100%
8.3	Probar recuperación de respaldo ex	Andree	1	1	0					100%
			ESTIMAR	COMPLETADO	RESTANTE	DÍAS	ESTI	DÍAS		
TOTAL DE HORAS			40	40	0	7	5.71429			
DÍA										

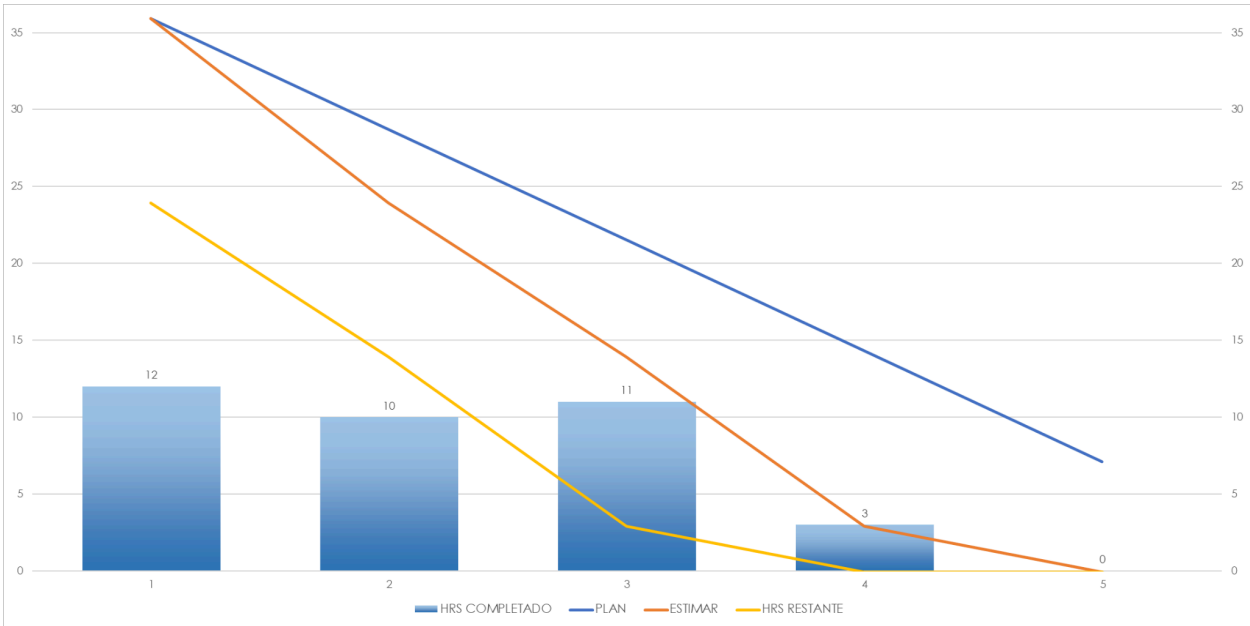
DÍA	1	2	3	4	5	6	7
PLAN	40	34	29	23	17	11	6
ESTIMAR	40	34	30	24	16	12	5
S COMPLETADO	6	4	6	8	4	7	5
HRS RESTANTE	34	30	24	16	12	5	0



Sprint 5.

ESTRUCTURA DE DESGLOSE DEL	TÍTULO DE LA TAREA	PROPIETARIO DE LA TAREA	CANTIDAD DE TRABAJO EN HORAS			CARRERA	FECHA DE INICIO	FECHA DE VENCIMIENTO	DURACIÓN	PCT DE TAREA COMPLETADA
			ESTIMAR	COMPLETADO	RESTANTE					
1	HU-20: Exportación de Datos Anonimizados (8h)		8	8	0		13/10/2025	17/10/2025	5	100%
1.1	Implementar exportación de registros	Yersson	4	4	0					100%
1.2	Asegurar eliminación de datos personales	Yersson	3	3	0					100%
1.3	Validar exportación correcta y legible	Yersson	1	1	0					100%
2	HU-21: Soporte Multilinguaje (8h)		8	8	0		13/10/2025	17/10/2025	5	100%
2.1	Crear estructura de archivos para traducción	Yersson	4	4	0					100%
2.2	Traducir textos clave de la interfaz	Yersson	3	3	0					100%
2.3	Implementar cambio dinámico de idioma	Yersson	1	1	0					100%
3	HU-30: Manual de Usuario (8h)		8	8	0		13/10/2025	17/10/2025	5	100%
3.1	Redactar guía detallada con pasos y capturas	Alejandro	4	4	0					100%
3.2	Incluir capturas de pantalla y ejemplos	Alejandro	2	2	0					100%
3.3	Publicar el manual en formato PDF	Alejandro	2	2	0					100%
4	HU-29: Documentación Técnica (12h)		12	12	0		13/10/2025	17/10/2025	5	100%
4.1	Documentar arquitectura general del sistema	Andree	4	4	0					100%
4.2	Describir dependencias, instalación y configuración	Andree	4	4	0					100%
4.3	Crear diagrama final del sistema y anexos	Andree	4	4	0					100%
DATOS DE DESGASTE			ESTIMAR	COMPLETADO	RESTANTE	DÍAS	EST/DÍAS	DÍA		
TOTAL DE HORAS			36	36	0	5	7.20000			

DÍA	1	2	3	4	5
PLAN	36	29	22	14	7
ESTIMAR	36	24	14	3	0
HRS COMPLETADO	12	10	11	3	0
HRS RESTANTE	24	14	3	0	0



7.3. Tablero Scrum – Kanban (Imagen y enlace público)

Sprint 1.

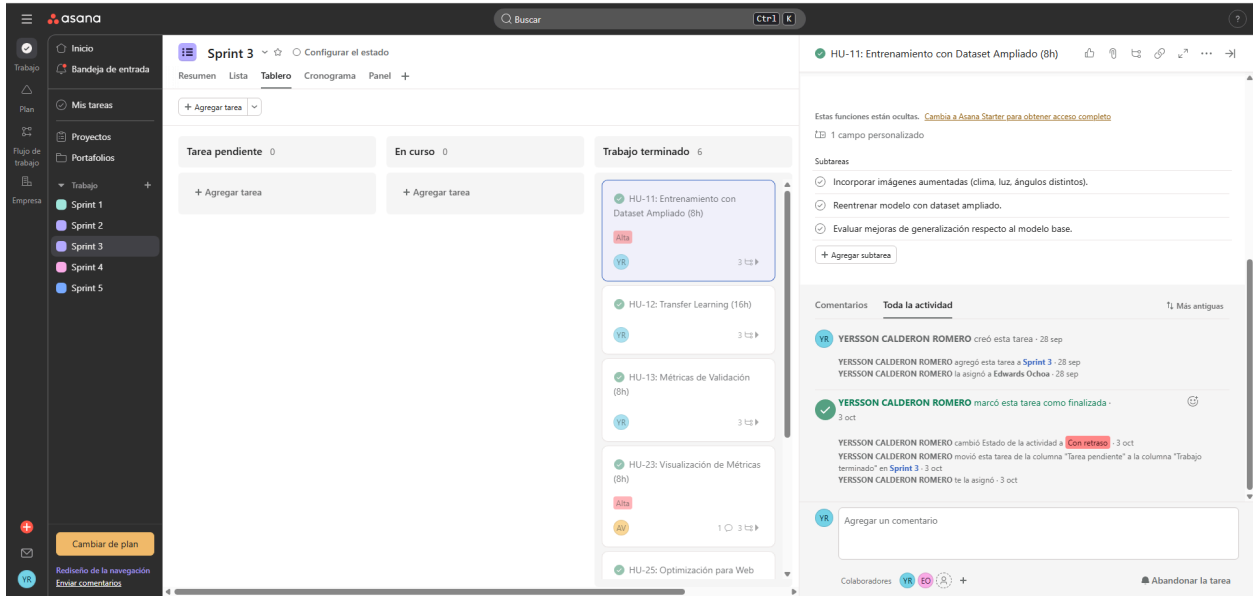
<https://app.asana.com/1/1110263497108613/project/1211417566709710/board/1211417623865614>

Sprint 2.

<https://app.asana.com/1/1110263497108613/project/1211417594384707/list/1211417654109355>

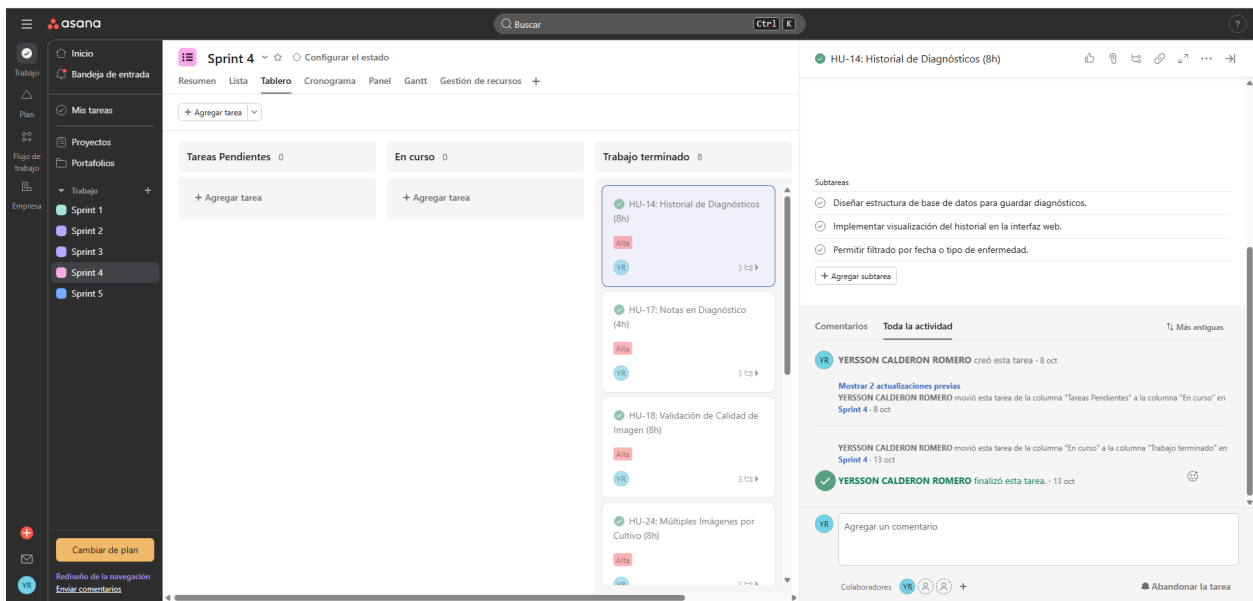
Sprint 3.

<https://app.asana.com/1/1110263497108613/project/1211488064071969/list/1211488414723182>



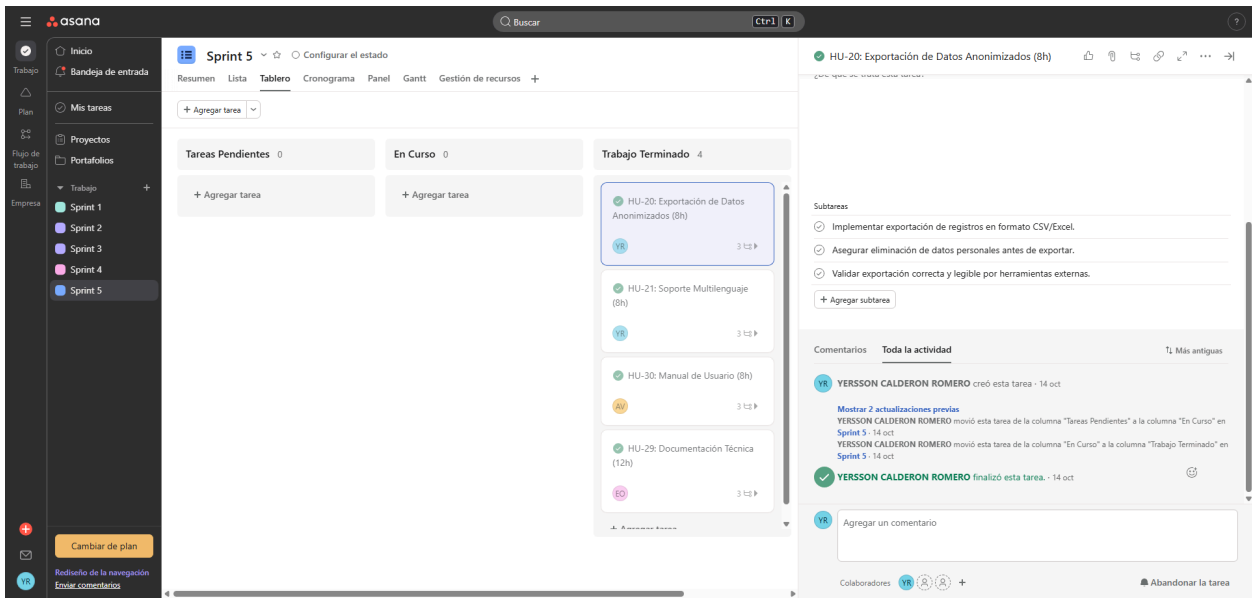
Sprint 4.

<https://app.asana.com/1/1110263497108613/project/1211593957703665/list/1211593881579449>



Sprint 5.

<https://app.asana.com/1/1110263497108613/project/1211644724450729/list/1211644604853759>



8. VIII. PRUEBAS Y RESULTADOS Y DISCUSIÓN

8.1. Pruebas por PMV

8.1.1. Prueba PMV 1

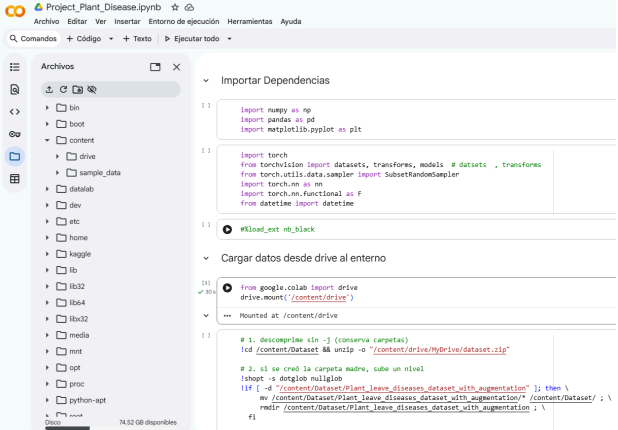
PMV1–HU1–Configuración del entorno

Implementación de sistema de visión por computadora con CNN para diagnóstico de enfermedades en hojas de cultivos andinos en comunidades de Cusco



Tablero Kanban

- [Sprint 1](#)
- [Sprint 2](#)
- [Sprint 3](#)
- [Sprint 4](#)
- [Sprint 5](#)
- [Paper](#)



```
Project_Plant_Disease.ipynb
Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda
Q Comandos + Código + Texto + Ejecutar todo
Archivos
bin
boot
content
drive
sample_data
datablab
dev
etc
home
kiaggle
lib
lib32
lib64
libx32
media
mnt
opt
proc
python-apt
74.52 GB disponibles
Importar Dependencias
1
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

2
import torch
from torchvision import datasets, transforms, models # datasets , transforms
from torch.utils.data.sampler import SubsetRandomSampler
import torch.nn as nn
import torch.nn.functional as F
from datetime import datetime

3
#load_ext nb_black

Cargar datos desde drive al entorno
11
from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive

1
# 1. descomprime sin -j (conserva carpetas)
!cd /content/Dataset && unzip -o "/content/drive/MyDrive/dataset.zip"

2. si se creó la carpeta madre, sube un nivel
!lsapt -o /content/multiglob
!if [ -d "/content/Dataset/Plant_Leaves_Diseases_dataset_with_augmentation" ]; then \
mv /content/Dataset/Plant_Leaves_Diseases_dataset_with_augmentation/* /content/Dataset/ ; \
rm -rf /content/Dataset/Plant_Leaves_Diseases_dataset_with_augmentation ; \
fi
```

Resultados.

- Python y librerías instaladas sin errores.
- GPU correctamente reconocida por PyTorch.
- Entorno virtual configurado y repositorio sincronizado.

Discusión.

La correcta configuración del entorno permitió iniciar el proyecto sin inconvenientes, garantizando la compatibilidad con la GPU y asegurando que las dependencias no interfirieran en el desarrollo. El repositorio en GitHub facilitó el control de versiones y la colaboración.

PMV1-HU2 – Recolección y organización del dataset

Data for: Identification of Plant Leaf Diseases Using a 9-layer Deep Convolutional Neural Network

Published: 17 April 2019 | Version 1 | DOI: 10.17632/tywbtsjrjv.1
Contributors: ARUN PANDIAN J, GEETHARAMANI GOPAL

Description

In this data-set, 39 different classes of plant leaf and background images are available. The data-set containing 61,486 images. We used six different augmentation techniques for increasing the data-set size. The techniques are image flipping, Gamma correction, noise injection, PCA color augmentation, rotation, and Scaling.

The classes are,
1.Apple_scab

Resultados

- Dataset público adecuado encontrado.
- Cultivos andinos correctamente definidos.
- Imágenes clasificadas y organizadas sin errores.

Discusión.

El éxito en la recolección y organización del dataset fue fundamental para entrenar el modelo. Seleccionar un dataset adecuado y asegurarse de que las imágenes estuvieran bien organizadas permitió que el proceso de entrenamiento fuera más eficiente. La definición clara de los cultivos andinos también fue esencial para garantizar que los diagnósticos fueran específicos y relevantes para la región de estudio.

PMV1-HU3-Preprocesamiento de imágenes

```
import torch, torchvision.transforms as T
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
transform = T.Compose([
    T.Resize(255),
    T.CenterCrop(224),
    T.ToTensor()
])

mean = 0.
std = 0.
nb_samples = 0.
for inputs, _ in train_loader:
    batch_samples = inputs.size(0)
    inputs = inputs.view(batch_samples, inputs.size(1), -1)
    mean += inputs.mean(2).sum(0)
    std += inputs.std(2).sum(0)
    nb_samples += batch_samples

mean /= nb_samples
std /= nb_samples

print(f"Mean of training data: {mean}")
print(f"Std of training data: {std}")

Mean of training data: tensor([0.4393, 0.4781, 0.3758])
Std of training data: tensor([0.1684, 0.1429, 0.1830])
```

Resultados.

- Imágenes normalizadas correctamente.
- Técnicas de aumento de datos aplicadas exitosamente.
- Dataset limpio y sin duplicados.

Discusión.

El preprocesamiento de las imágenes garantizó que los datos fueran adecuados para el entrenamiento del modelo. Al normalizar las imágenes y aplicar técnicas de aumento de datos, mejoramos la robustez del modelo y reducimos el riesgo de sobreajuste. La limpieza del dataset eliminó imágenes innecesarias o duplicadas, mejorando la calidad del entrenamiento.

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 32, 224, 224]	896
ReLU-2	[-1, 32, 224, 224]	0
BatchNorm2d-3	[-1, 32, 224, 224]	64
Conv2d-4	[-1, 32, 224, 224]	9,248
ReLU-5	[-1, 32, 224, 224]	0
BatchNorm2d-6	[-1, 32, 224, 224]	64
MaxPool2d-7	[-1, 32, 112, 112]	0
Conv2d-8	[-1, 64, 112, 112]	18,496
ReLU-9	[-1, 64, 112, 112]	0
BatchNorm2d-10	[-1, 64, 112, 112]	128
Conv2d-11	[-1, 64, 112, 112]	36,928
ReLU-12	[-1, 64, 112, 112]	0
BatchNorm2d-13	[-1, 64, 112, 112]	128
MaxPool2d-14	[-1, 64, 56, 56]	0
Conv2d-15	[-1, 128, 56, 56]	73,856
ReLU-16	[-1, 128, 56, 56]	0
BatchNorm2d-17	[-1, 128, 56, 56]	256
Conv2d-18	[-1, 128, 56, 56]	147,584
ReLU-19	[-1, 128, 56, 56]	0
BatchNorm2d-20	[-1, 128, 56, 56]	256
MaxPool2d-21	[-1, 128, 28, 28]	0
Conv2d-22	[-1, 256, 28, 28]	295,168
ReLU-23	[-1, 256, 28, 28]	0
BatchNorm2d-24	[-1, 256, 28, 28]	512
Conv2d-25	[-1, 256, 28, 28]	590,080
ReLU-26	[-1, 256, 28, 28]	0
BatchNorm2d-27	[-1, 256, 28, 28]	512
MaxPool2d-28	[-1, 256, 14, 14]	0
Dropout-29	[-1, 50176]	0
Linear-30	[-1, 1024]	51,381,248
ReLU-31	[-1, 1024]	0
Dropout-32	[-1, 1024]	0
Linear-33	[-1, 39]	39,975
Total params: 52,595,399		
Trainable params: 52,595,399		
Non-trainable params: 0		

Resultados

- Arquitectura de la red definida sin errores.
- Entrenamiento inicial completado con éxito.
- Métricas preliminares generadas.

Discusión.

El diseño de la arquitectura base fue un paso crucial en el proyecto. Al definir las capas adecuadas y configurar correctamente la función de pérdida y optimizador, pudimos realizar un entrenamiento inicial sin problemas. Las métricas preliminares proporcionaron una primera evaluación del desempeño del modelo y nos permitieron ajustar la arquitectura para mejorar los resultados.

PMV1–HU5–Clasificación básica

--

PMV1–HU8–Mostrar nivel de confianza

8.1.2. Prueba PMV 2

PMV2-HU6 – Detección de cultivo

PMV2-HU7 – Diagnóstico por enfermedad

PMV2-HU9 – Interfaz web

--

PMV2-HU10 – Subida intuitiva

PMV2-HU15 – Ejemplos visuales

PMV2-HU16 – Recomendaciones básicas

8.1.3. Prueba PMV 3

PMV3-HU12 – Transfer Learning

--

PMV3-HU13 – Métricas de validación

PMV3-HU23 – Visualización de métricas

PMV3-HU25 – Optimización para Web

PMV3-HU27 – Validación externa

PMV3-HU11 – Dataset ampliado

PMV3-HU14-Historial de Diagnósticos

PMV3-HU17-Notas en Diagnóstico

PMV3-HU18-Validación de Calidad de Imagen

--

PMV3-HU24-Múltiples Imágenes por Cultivo

PMV3-HU26-Alertas Tempranas

PMV3-HU19-Clasificación de Severidad

PMV3-HU22-Retroalimentación del Usuario
--

PMV3-HU28-Backup Automático

PMV3-HU20-Exportación de Datos Anonimizados

PMV3-HU21-Soporte Multilenguaje

PMV3-HU30-Manual de Usuario

PMV3-HU29-Documentación Técnica

9. IX. LECCIONES APRENDIDAS (MIN 3) por PMV

9.1. Prueba PMV 1

Importancia de la calidad de los datos: Durante el proceso de entrenamiento, se aprendió que las imágenes de baja calidad, como aquellas mal iluminadas o borrosas, pueden afectar significativamente la precisión del modelo. El preprocesamiento, como la normalización y el aumento de datos, fue fundamental para minimizar este impacto y mejorar la generalización del modelo.

Optimización de los parámetros: Se descubrió que ajustar los hiperparámetros de la CNN, como el número de capas y los filtros, es esencial para alcanzar un buen desempeño. A través de la experimentación, logramos una arquitectura eficiente que evitó el sobreajuste, asegurando que el modelo no solo aprendiera de manera precisa, sino también generalizara bien en datos no vistos.

Desafíos con la capacidad de cómputo: Aunque el uso de GPU aceleró el proceso de entrenamiento, el tamaño del dataset y la complejidad del modelo requirieron optimizaciones adicionales en términos de uso de memoria y tiempo de entrenamiento, lo que nos llevó a experimentar con técnicas de poda de redes y reducción de tamaño de modelos.

9.2. Prueba PMV 2

Desarrollo de la API: Una de las lecciones clave fue comprender la importancia de crear un pipeline de inferencia robusto. Aunque el modelo entrenado funcionaba bien en un entorno controlado, al integrarlo en una API surgieron discrepancias en el rendimiento, principalmente debido a diferencias en el preprocesamiento de las imágenes en el entrenamiento y la inferencia. La solución fue estandarizar el pipeline para asegurar consistencia.

Escalabilidad y eficiencia: A medida que comenzamos a implementar la API para hacer predicciones en tiempo real, descubrimos la necesidad de optimizar la latencia y los tiempos de respuesta. La reducción del tamaño del modelo y la compresión de los archivos fueron esenciales para garantizar un buen desempeño en entornos de producción.

Validación del modelo en producción: Aunque el modelo fue validado previamente en un conjunto de datos de entrenamiento, se descubrió que en producción, con imágenes del mundo real, algunas predicciones no eran tan precisas. Esto subrayó la importancia de realizar pruebas continuas en el entorno de producción para ajustar y recalibrar el modelo según sea necesario.

9.3. Prueba PMV 3

Simplicidad y accesibilidad: Una lección valiosa fue que la simplicidad en el diseño de la interfaz es crucial para la adopción de usuarios. Los agricultores, que en su mayoría no son expertos en tecnología, prefieren una interfaz sencilla que no les sobrecargue de información y les permita recibir los resultados de manera rápida y clara.

Pruebas en dispositivos de bajo rendimiento: Al probar la interfaz web en dispositivos móviles de bajo costo, se observó que el sistema debía ser optimizado para funcionar en teléfonos con recursos limitados. La optimización de la interfaz y el modelo para soportar dispositivos de bajo rendimiento mejoró la accesibilidad, asegurando que la tecnología fuera útil en las zonas rurales.

Validación del modelo con usuarios finales: La validación con los agricultores mostró que los resultados eran confiables, pero algunos usuarios sugerían mejorar la visualización de los diagnósticos. Incorporar ejemplos visuales de hojas sanas y enfermas, junto con el diagnóstico, facilitó la comprensión y la confianza en el sistema, lo que incrementó su aceptación.

10. X. CONCLUSIONES

El modelo propuesto ha logrado establecer una solución eficaz para detectar enfermedades en las hojas de cultivos andinos, con una precisión adecuada para apoyar la toma de decisiones en el ámbito agrícola. La implementación de técnicas de aprendizaje

profundo ha permitido un diagnóstico temprano y fiable, lo cual es crucial para mitigar pérdidas agrícolas en la región Cusco, promoviendo la seguridad alimentaria y la sostenibilidad.

El desarrollo e implementación de un modelo CNN utilizando PyTorch fue un paso fundamental para la clasificación precisa de enfermedades. La arquitectura entrenada mostró un rendimiento destacado, permitiendo identificar de manera efectiva las enfermedades en cultivos andinos, lo cual es un avance importante en el uso de inteligencia artificial en la agricultura.

El entrenamiento con un dataset balanceado ha permitido una adecuada generalización del modelo, reduciendo los riesgos de sobreajuste. Al utilizar imágenes reales de cultivos andinos, se garantizó que el modelo estuviera alineado con las condiciones del mundo real, mejorando así su fiabilidad en situaciones prácticas.

El diseño de la interfaz web interactiva ha sido clave para facilitar el acceso al diagnóstico por parte de los agricultores. Su diseño sencillo y funcional permite que los usuarios suban imágenes de manera rápida y reciban diagnósticos inmediatos, lo que agiliza la toma de decisiones y mejora la eficiencia en la gestión de cultivos.

La evaluación del rendimiento del modelo utilizando métricas de precisión, recall y F1-score, junto con la matriz de confusión, ha sido fundamental para asegurar la fiabilidad del sistema. Estas métricas proporcionaron información detallada sobre el desempeño del modelo, permitiendo realizar ajustes y mejorar continuamente la precisión del diagnóstico.

11. XI REFERENCIAS BIBLIOGRÁFICAS

1. PAL, Chiranjit, Swastik KARMAKAR, Imon MUKHERJEE y Partha Pratim CHAKRABARTI. *A lightweight and explainable CNN model for empowering plant disease diagnosis*. En línea. Scientific Reports, vol. 15 (agosto de 2025), n.º 1. ISSN 2045-2322. Disponible en: <https://doi.org/10.1038/s41598-025-94083-1>. [consultado el 24/10/2025].
2. MOHANTY, Sharada P., David P. HUGHES y Marcel SALATHÉ. *Using deep learning for image-based plant disease detection*. En línea. Frontiers in Plant Science, vol. 7 (septiembre de 2016). ISSN 1664-462X. Disponible en: <https://doi.org/10.3389/fpls.2016.01419>. [consultado el 24/10/2025].
3. PACAL, Ishak, Ismail KUNDURACIOGLU, Mehmet Hakki ALMA, Muhammet DEVECI, Seifedine KADRY et al. *A systematic review of deep learning techniques for plant diseases*. En línea. Artificial Intelligence Review, vol. 57

- (septiembre de 2024), n.º 11. ISSN 1573-7462. Disponible en: <https://doi.org/10.1007/s10462-024-10944-7>. [consultado el 24/10/2025].
4. KRISHNA, Manjunatha Shettigere, Pedro MACHADO, Richard I. OTUKA, Salisu W. YAHAYA, Filipe NEVES DOS SANTOS et al. *Plant leaf disease detection using deep learning: a multi-dataset approach*. En línea. J, vol. 8 (enero de 2025), n.º 1, p. 4. ISSN 2571-8800. Disponible en: <https://doi.org/10.3390/j8010004>. [consultado el 24/10/2025].
 5. *Millions of global agricultural crops are lost to pests annually- FAO - Prensa Latina*. En línea. Prensa Latina - Latin American News Agency. [s. f.]. Disponible en: https://www.plenglish.com/news/2023/05/12/millions-of-global-agricultural-crops-are-lost-to-pests-annually-fao/?utm_source. [Consultado el 24/10/2025].
 6. SAVARY, Serge, Laetitia WILLOCQUET, Sarah Jane PETHYBRIDGE, Paul ESKER, Neil MCROBERTS et al. *The global burden of pathogens and pests on major food crops*. En línea. Nature Ecology & Evolution, vol. 3 (febrero de 2019), n.º 3, pp. 430–439. ISSN 2397-334X. Disponible en: <https://doi.org/10.1038/s41559-018-0793-y>. [Consultado el 24/10/2025].
 7. LEON, Betsabe, Rosario BRAVO, Joven LLANOS, Alicia LEON y Wenceslao MEDINA. *Fluctuation of main insect pests and diseases in cañihua (Chenopodium pallidicaule Aellen) of the Peruvian Andean Altiplano*. En línea. Revista de la Facultad de Agronomía, Universidad del Zulia, vol. 39 (marzo de 2022), n.º 2, e223923. ISSN 2477-9407. Disponible en: [https://doi.org/10.47280/revfacagron\(luz\).v39.n2.01](https://doi.org/10.47280/revfacagron(luz).v39.n2.01). [Consultado el 24/10/2025].
 8. *Senasa realiza vigilancia fitosanitaria en cultivos de papa de región Cusco*. En línea. Agencia Peruana de Noticias | ANDINA. [s. f.]. Disponible en: <https://andina.pe/agencia/noticia-senasa-realiza-vigilancia-fitosanitaria-cultivos-papa-region-cusco-665596.aspx>. [Consultado el 24/10/2025].
 9. Comisión Europea. «Qué es la Inteligencia Artificial». 19 abr 2023. Disponible en: https://planderecuperacion.gob.es/noticias/que-es-inteligencia-artificial-ia-prtr?utm_source [consultado 24/10/2025].
 10. Techopedia. «¿Qué significa inteligencia artificial y su definición?». Disponible en: https://www.techopedia.com/es/definicion/inteligencia-artificial-ai?utm_source [consultado 24/10/2025].

11. Techopedia. «¿Qué es Deep Learning? Definición y distintos tipos». Disponible en: https://www.techopedia.com/es/definicion/deep-learning?utm_source [consultado 24/10/2025].
12. TechTarget. «What is a Convolutional Neural Network (CNN)?». Disponible en: https://www.techtarget.com/searchenterpriseai/definition/convolutional-neural-network?utm_source [consultado 24/10/2025].
13. Deepchecks Glossary. «Convolutional Neural Network». Disponible en: https://www.deepchecks.com/glossary/convolutional-neural-network/?utm_source [consultado 24/10/2025].
14. Aiphabet. «CNN layers». Disponible en: https://www.aiphabet.org/learn/introduction-to-ai/neural-networks/convolutional-neural-networks/article%3Acnn-layers?utm_source [consultado 24/10/2025].
15. IBM Think. «¿Qué es el aprendizaje por transferencia?». Disponible en: https://www.ibm.com/es-es/think/topics/transfer-learning?utm_source [consultado 24/10/2025].
16. Díaz, R. «Métricas de clasificación: aprende a evaluar tu modelo». The Machine Learners, 2025. Disponible en: <https://www.themachinelearners.com/metricas-de-clasificacion/> [consultado 24/10/2025].
17. Murel, J. «¿Qué es una matriz de confusión?». IBM Think, 2025. Disponible en: <https://www.ibm.com/mx-es/think/topics/confusion-matrix> [consultado 24/10/2025].
18. Real Academia Española. «fitopatología». Diccionario de la lengua española. 2025. Disponible en: <https://dle.rae.es/fitopatolog%C3%ADa> [consultado 24/10/2025].
19. Todomicología. «La fitopatología: su importancia crucial en la agricultura». 2025. Disponible en: <https://todomicologia.com/patogenos/la-fitopatologia-su-importancia-crucial-en-la-agricultura/> [consultado 24/10/2025].
20. EOS. «Enfermedades de las plantas: tipos, causas y control». 2025. Disponible en: <https://eos.com/es/blog/enfermedades-de-las-plantas/> [consultado 24/10/2025].
21. OpenCV. «Computer Vision and Image Processing: A Beginner's Guide». 2025. Disponible en: <https://opencv.org/blog/computer-vision-and-image-processing/> [consultado 24/10/2025].
22. GeeksforGeeks. «What is Image Classification?». 2025. Disponible en: <https://www.geeksforgeeks.org/computer-vision/what-is-image-classification/> [consultado 24/10/2025].

Anexo 1: Repositorio del proyecto en GitHub

<https://github.com/Taller-de-proyectos-I/plantandes.git>